COMP1023: Introduction to Python Programming

WU Jinjun

November 13, 2026

Contents

1 Introduction to Computers and Python **6**
1.1 What Is a Computer? . . . 6
1.2 Bits, Bytes, and Storage . . . 6
1.3 Problem Solving and Algorithms . . . 6
1.4 Computer Programs and Programming . . . 7
1.5 Programming Languages . . . 8
1.5.1 Compiler vs. Interpreter . . . 8
1.6 A Brief History of Python . . . 8
1.6.1 Why Python? . . . 9
1.7 The Development Environment . . . 9
1.8 Types of Programming Errors . . . 9
1.9 Your First Python Program . . . 9

2 Basic Programming Language Elements **10**
2.1 Writing a Simple Python Program . . . 10
2.2 Comments . . . 11
2.3 The main() Function . . . 11
2.3.1 With main() vs. Without main() . . . 11
2.4 Kinds of Statements . . . 12
2.5 Data Types . . . 12
2.6 Literals . . . 12
2.7 Booleans: True and False . . . 13
2.8 Strings . . . 13
2.9 Objects, Variables, and References . . . 13
2.9.1 Rules for Choosing Variable Names . . . 14
2.10 Dynamic Typing and Type Hinting . . . 14
2.11 Named Constants . . . 14
2.12 Explicit Type Conversions . . . 15
2.13 Python Convention and Good Programming Style . . . 15
2.14 Mutability of Types . . . 15
2.15 Random Number Generation . . . 15
2.16 Output: The print() Function . . . 16
2.16.1 Formatted String Literals (f-strings) . . . 16

2.16.2 Printing Different Types and Multiline Text . . . 17
2.17 Input: The input() Function . . . 17
2.18 Indentation . . . 18
2.19 Good Programming Style . . . 18

3 Python Basic Operations **19**
3.1 Arithmetic Operations . . . 19
3.2 Types of the Results . . . 19
3.3 Integer Division (//) and Exponentiation (**) . . . 19
3.4 Modulo Operation (%) . . . 20
3.5 Subtraction and Negation, Division by Zero . . . 20
3.6 Expressions and Precedence . . . 20
3.7 Assignment Operations . . . 21
3.7.1 Augmented Assignment . . . 21
3.8 Relational (Comparison) Operations . . . 22
3.8.1 String Comparisons . . . 22
3.9 Logical Operations . . . 22
3.9.1 Short-Circuiting (Lazy) Evaluation . . . 23
3.10 Identity and Membership Operations . . . 23
3.11 Associativity and Operator Precedence . . . 23
3.12 Floating-Point Errors . . . 24
3.13 Common Pitfalls . . . 24

4 Branching (Selection Statements) **24**
4.1 Boolean Expressions and Operators . . . 25
4.2 The if Statement . . . 25
4.3 The if-else Statement . . . 26
4.4 The if-elif-else Statement (if-else Ladder) . . . 26
4.5 Nested if Statements . . . 27
4.6 Truthiness . . . 27
4.7 Conditional Expression (Ternary Operator) . . . 27
4.8 match-case Statement . . . 28
4.8.1 Type Patterns (Extra) . . . 28

5 Looping **29**
5.1 Components of a Loop . . . 30
5.2 The while Loop . . . 30
5.2.1 Sentinel-Controlled Loop . . . 31
5.3 The for Loop . . . 31
5.3.1 The range() Function . . . 31
5.4 while versus for . . . 32
5.5 The break Statement . . . 32
5.5.1 Executing a Loop At Least Once . . . 33
5.6 The continue Statement . . . 33
5.7 The else Clause of while and for . . . 33
5.8 Nested Loops . . . 34
5.9 Variable Scope in Loops . . . 35

6 Collections: Container Data Types (Part I) 36
6.1 Collections in Python . . . 36
6.2 Why Lists? . . . 36
6.3 List Basics . . . 36
6.4 Operations for Sequences . . . 37
6.4.1 Comparison Operators for Lists . . . 37
6.5 The Index Operator . . . 38
6.6 List Slicing . . . 38
6.6.1 Default Values and Edge Cases . . . 38
6.7 Traversing a List . . . 39
6.8 List Comprehensions . . . 39
6.9 List Methods . . . 39
6.10 Splitting a String into a List . . . 40
6.11 Inputting Lists . . . 40
6.12 Copying Lists . . . 41
6.13 Two-Dimensional Lists . . . 41

7 Functions (Part 1) 43
7.1 Why Functions? . . . 43
7.2 Defining a Function . . . 43
7.3 The Flow of a Function Call . . . 44
7.4 Type Hints in Functions . . . 44
7.5 Functions That Return `None` . . . 45
7.6 Positional and Keyword Arguments . . . 45
7.7 Default Parameter Values . . . 46
7.8 Returning Multiple Values . . . 46
7.9 The `pass` Statement . . . 47

8 Functions (Part 2) 47
8.1 Scope of Variables . . . 47
8.2 The `global` Statement . . . 48
8.3 Name Conflicts . . . 49
8.4 Memory: Stack Frame, Global Frame, and Heap . . . 49
8.5 Modularizing Code . . . 49
8.6 Passing Lists to Functions . . . 50
8.7 Variadic Arguments: `*args` and `**kwargs` . . . 51
8.8 Lambda Functions . . . 51
8.9 Common Use Cases: `filter`, `map`, and `reduce` . . . 51

9 Collections: Container Data Types (Part II) 52
9.1 Tuples . . . 52
9.1.1 Functions for Tuples . . . 53
9.1.2 Indexing, Immutability, and Mutable Elements . . . 53
9.1.3 Tuple Slicing . . . 54
9.1.4 Traversing, Comprehensions, and Methods . . . 54
9.1.5 Copying Tuples, Two-Dimensional Tuples, and Packing . . . 54
9.2 Dictionaries . . . 55

9.2.1 Adding, Modifying, Retrieving, and Deleting Items 56
9.2.2 Traversing a Dictionary 56
9.2.3 Dictionary Comprehensions 57
9.2.4 Dictionary Methods 57
9.2.5 Copying Dictionaries 58
9.3 Choosing a Container Type 58

10 Recursion **59**
10.1 Introduction 59
10.2 The Handshake Problem 59
10.3 General Structure of a Recursive Function 59
10.4 The Factorial Problem 60
10.5 Fibonacci's Bunnies 60
10.6 Recursion and Loops 61
10.7 Stack Overflow and Its Handling 61
10.8 Trade-offs and Caution 61

11 NumPy **62**
11.1 Why NumPy 62
11.2 Creating Arrays and Attributes 63
11.3 Indexing and Negative Indexing 63
11.4 Slicing 64
11.5 Integer Array (Fancy) Indexing 64
11.6 Boolean Masking 65
11.7 Reshaping and Expanding Dimensions 65
11.8 View vs Copy 65
11.9 Vectors, Matrices and Tensors 66
11.10 Dot Product and Matrix Multiplication 66
11.11 Transpose 67
11.12 Broadcasting 67

12 Object-Oriented Programming **68**
12.1 Everything Is an Object 68
12.2 Objects and Classes 68
12.3 Defining a Class and __init__ 68
12.4 Instance Attributes vs Class Attributes 69
12.5 UML Class Diagrams 69
12.6 Public, Protected and Private Members 70
12.7 Name Mangling 71
12.8 Accessors and Mutators 71
12.9 Assignment Copies the Reference, Not the Object 71

13 Pandas **72**
13.1 Introduction 72
13.2 The Series 73
13.2.1 Creating a Series 73
13.2.2 Accessing and Slicing 73
13.2.3 Masking 74

13.3 The DataFrame . . . 74
13.3.1 Creating a DataFrame . . . 74
13.3.2 Inspecting a DataFrame . . . 75
13.3.3 Selecting Columns, Rows and Cells . . . 75
13.4 Handling Missing Values . . . 76
13.5 Grouping and Aggregation . . . 76
13.6 Reading and Writing CSV Files . . . 77

14 Matplotlib **78**
14.1 Introduction . . . 78
14.2 The Pyplot and Object-Oriented Interfaces . . . 78
14.3 Figures, Axes and Axis . . . 78
14.4 Pairwise Data Visualization . . . 79
14.4.1 Line Plots with plot . . . 79
14.4.2 Scatter Plots . . . 80
14.4.3 Subplots . . . 81
14.4.4 A Sequence of Points or Segments . . . 81
14.4.5 Bar Charts . . . 82
14.5 Statistical Distributions . . . 83
14.5.1 Histograms . . . 83
14.5.2 Box Plots . . . 84
14.6 Contour Plots . . . 84
14.7 Filling the Area Between Curves . . . 86
14.8 Saving a Figure to a File . . . 87
14.9 Reading Data From Files . . . 87

1 Introduction to Computers and Python

1.1 What Is a Computer?

A computer consists of both **hardware** and **software**.

Hardware vs. Software

Hardware is the visible, physical part of the computer: the CPU, main memory (RAM), storage devices, input/output devices, and communication devices, all interconnected by a **bus**. **Software** is the invisible set of instructions that control the hardware and enable it to perform specific tasks.

Main hardware components:

- **CPU** – executes instructions.
- **Main Memory (RAM)** – temporary storage for running programs.
- **Storage Devices** – e.g., hard drives and SSDs.
- **Input Devices** – e.g., keyboard, mouse.
- **Output Devices** – e.g., monitors, printers.
- **Communication Devices** – e.g., network interface cards.

Memory is an ordered sequence of bytes; each byte has a unique **address** used to locate it for storing and retrieving data.

1.2 Bits, Bytes, and Storage

Information (data and programs) is stored as sequences of 0s and 1s, called **bits** (binary digits). A **byte** is 8 bits, and storage capacity is measured in bytes and their multiples:

Unit	Size
bit	a single binary digit, 0 or 1
byte	8 bits
KB	1,024 bytes
MB	1,024 KB
GB	1,024 MB
TB	1,024 GB

Powers of 1,024, Not 1,000

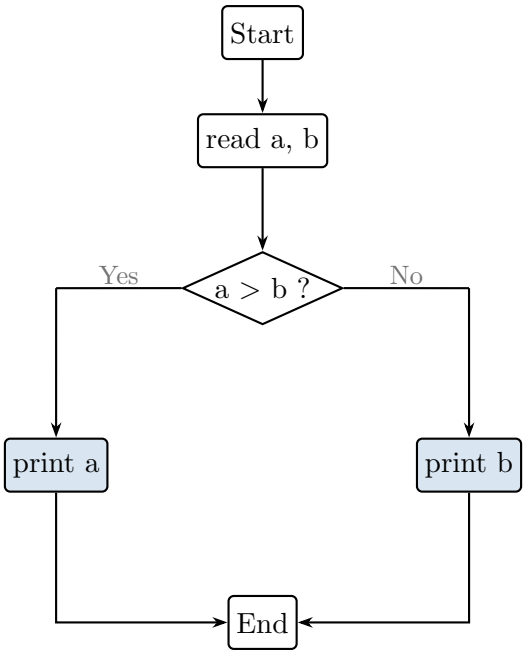
A kilobyte is 1,024 bytes, because the units are built from powers of two ($2^{10} = 1{,}024$). Do not confuse these with the decimal prefixes used by disk manufacturers.

1.3 Problem Solving and Algorithms

Algorithm

An **algorithm** is a step-by-step description of how to accomplish a task. It is the idea behind the solution, independent of any programming language.

An algorithm can be represented as a **flowchart**, as **pseudocode**, or as a **computer program**. In the retail-discount example, a valid membership card charges 80% of the item price, otherwise the full price.



The same algorithm as pseudocode:

```
READ Membership Card No.
READ ItemPrice
IF Membership Card No. is valid THEN
  price = ItemPrice * 0.8;
ELSE
  price = ItemPrice;
END IF
```

A Program Is Not the Algorithm

Different programmers may write the same algorithm in different languages and styles, so a single program is not a good language-independent representation of the algorithm – the flowchart or pseudocode is.

1.4 Computer Programs and Programming

Computer Program

A **computer program** is a set of step-by-step instructions that performs specific tasks to solve a problem (e.g., games, word processors, web browsers). **Computer programming** is the design and implementation of such programs.

Written in Python, the retail example becomes:

```
# Filename: sample_python_program.py
def is_valid(num):
    valid = False
    if num > 0 and num <= 100:
        valid = True
    return valid

card_no = int(input("Enter the card number: "))
item_price = int(input("Enter the item price: "))
price = 0
```

```
if is_valid(card_no):
    price = item_price * 0.8
else:
    price = item_price
print("The price is", price)
```

1.5 Programming Languages

Two issues matter when writing a program:

1. **Program syntax** – is the grammar of the instructions correct?
2. **Program logic** – is the program able to solve the problem?

High-Level Languages

High-level languages are user-friendly and abstract away most hardware details. They fall into two groups: **procedural / structured** languages (e.g., COBOL, Pascal, C) and **object-oriented** languages (e.g., Python, C++, Java).

1.5.1 Compiler vs. Interpreter

A program in a high-level language is called **source code**. A **compiler** translates the entire source code into a machine-code file, which is then executed. An **interpreter** reads one statement, executes it immediately, and may translate it to an intermediate or machine representation before moving to the next statement.

	Compiler	**Interpreter**
Translation	Entire source code at once	One statement at a time
Execution	Runs after full translation	Executes immediately, line by line
Output	A machine-code file	No separate machine-code file
Speed	Compiled programs run faster	Interpreted programs run slower

Quick Check

Fill in the blanks: a _____ is a binary digit; a _____ is 8 bits; a _____ translates all the source code before execution; an _____ translates and executes one statement at a time. (Answers: bit, byte, compiler, interpreter.)

1.6 A Brief History of Python

Python was created by **Guido van Rossum** at the National Research Institute for Mathematics and Computer Science in the Netherlands.

- **1991**: Python 1.0 released to the public.
- **1990s**: gained traction as a general-purpose language (web development, scientific computing, data analysis).
- **Major versions**: Python 2.0 (2000) and Python 3.0 (2008).
- **2010s**: surged in data science, machine learning, and AI.
- **2020s**: one of the most widely used languages today.

Python 2.x is no longer supported (since January 1, 2020), and **Python 3.x** is not backwards compatible with it. The latest stable version is Python 3.13, and this course uses Python SDK 3.13.

1.6.1 Why Python?

- **Easy to learn**: simple syntax, beginner-friendly.
- **Interpreted**: runs line by line, good for testing snippets.
- **Versatile**: web development, data analysis, AI, and more.
- **Large standard library**: rich functionality out of the box.
- **Cross-platform**: runs on Windows, macOS, and Linux.

1.7 The Development Environment

Two components are needed to write Python programs:

1. A machine with the **Python Software Development Kit (SDK)** installed (version 3.13 here). It includes the Python interpreter, the standard library, IDLE, the package installer `pip`, the documentation, and Python tools.
2. A **Python Integrated Development Environment (IDE)**: software with a code editor and interpreter integration. Examples: VS Code (used here), PyCharm, Jupyter Notebook, Atom, Sublime Text, Spyder.

The development cycle has two steps: (1) write the source code in an editor and save it with a `.py` extension; (2) run the file, returning to step 1 if there are any syntax or runtime errors. Under the hood, bytecode compilation generates `.pyc` files that are executed by the **PVM** (Python Virtual Machine); **PI** stands for Python Interpreter.

Command Line vs. IDE

IDEs let you run programs by clicking icons, but learn command-line execution too, since not all machines have an IDE. On Windows, open a command prompt with `Windows+R`, change folder with `cd`, then run `python welcome_students.py`. Check your version with `python --version`, since a bare `python` may call Python 2.

1.8 Types of Programming Errors

The Three Kinds of Errors

- **Syntax errors**: the program breaks the syntax rules of the language – e.g., a missing quotation mark or a misspelled keyword.
- **Runtime errors**: occur during execution when Python hits an operation it cannot perform – e.g., division by zero.
- **Logic errors**: the program runs but does not produce the desired outcome or behaves unexpectedly.

Spotting the Difference

A syntax error stops the program before it runs, a runtime error stops it while it runs, and a logic error lets it finish with a wrong answer – the hardest to find.

1.9 Your First Python Program

The example from the slides:

```
"""
First Python Program to print a text
"Welcome to COMP 1023!" on the screen
"""
# Filename: welcome_students.py
```

```
def main():
    print("Welcome to COMP 1023")

if __name__ == "__main__":
    main()
```

- The triple-quoted text is a **docstring**; `#` starts a comment (here recording the file name).
- `print(...)` displays text on the screen.
- Wrapping the logic in `def main()` and calling it under the `if __name__ == "__main__":` guard is the standard structure used throughout the course: the code runs only when the file is executed directly.

Case-Sensitive Language

Python is **case-sensitive**: it treats lower- and upper-case letters differently. `Print` is not `print`, and `Main` is not `main`. Writing `Print(...)` raises a `NameError`, so match the exact spelling and case of built-in names.

Summary

- A computer is **hardware** (CPU, memory, storage, I/O, communication devices joined by a bus) plus **software**.
- Information is stored as **bits**; 8 bits form a **byte**; storage uses KB, MB, GB, and TB (powers of 1,024).
- An **algorithm** is a step-by-step solution, shown by flowcharts, pseudocode, or programs.
- A **compiler** translates all source code before execution; an **interpreter** does it line by line, so compiled programs are faster.
- Python was created by **Guido van Rossum**; this course uses Python 3.13, which is not backwards compatible with Python 2.x.
- Writing Python needs the **SDK** and an **IDE**; files use the `.py` extension.
- Errors are **syntax**, **runtime**, and **logic** errors.
- A first program uses `print(...)`, comments/docstrings, and the `if __name__ == "__main__":` guard; Python is case-sensitive.

2 Basic Programming Language Elements

2.1 Writing a Simple Python Program

A first example sets a circle's radius, computes the area with the formula $area = PI \times radius \times radius$, and displays the result.

```
'''
A simple program to set the circle's radius to 20, compute the area
   using the formula:
area = PI * radius * radius, and display the result on the screen.
'''
# Filename: compute_area.py
def main():
    # Define the radius of the circle
    radius = 20
    # Define PI as 3.14159
    PI = 3.14159
    # Compute area
    area = PI * radius * radius
```

```
    # Display results
    print("The area for the circle of radius", radius, "is", area)

if __name__ == "__main__":
    main()
```

Output: `The area for the circle of radius 20 is 1256.636`.

2.2 Comments

Adding comments increases a program's readability. Comments are **not** treated as computer instructions: they are only for humans to read. Python provides two ways to add them.

- The text after a hash symbol `#` on a line is a comment (single-line).
- All lines between triple quotes (`'''` or `"""`) are treated as a comment (multi-line).

```
# Single line comment
'''
Multiple line comment
Comment line 1
...
Comment line n
'''
```

Prefer # for Comments

Triple quotes can render multiline strings, docstrings, and comments. For an actual comment it is clearer to use the `#` symbol, and to reserve triple quotes for docstrings and multiline strings.

2.3 The main() Function

`main()` is the **entry point** of a Python program or script: the designated function where the core logic resides. Defining a `main` function is good practice, although it is not mandatory.

```
def main():   # def means define
    # Statement(s)
    # ...

if __name__ == "__main__":
    main()
```

When a program is run (for example, `python my_program.py`), Python executes the code from top to bottom. Upon reaching the line `if __name__ == "__main__":`, Python checks the value of `__name__`. If it equals `"__main__"`, it calls `main()` and executes the code inside it.

2.3.1 With main() vs. Without main()

```
# Filename: compute_area1.py
def main():
    radius = 20     # radius is now 20
    PI = 3.14159    # PI is 3.14159
    area = PI * radius * radius
    print("The area for the circle of radius", radius, "is", area)
```

```
if __name__ == "__main__":
    main()

# Filename: compute_area2.py
radius = 20     # radius is now 20
PI = 3.14159    # PI is 3.14159
area = PI * radius * radius
print("The area for the circle of radius", radius, "is", area)
```

Running the two files directly shows no difference, but the difference becomes apparent in specific scenarios later in the course.

2.4 Kinds of Statements

Statements fall into three groups:

1. **Typical statements**: object creation and initialization, assignments, and function calls.
2. **Branching (selective) statements**: statements that select which statements to execute, such as `if..else` and `match..case`.
3. **Looping (iterative) statements**: statements that repeatedly execute code, such as `for` and `while`.

2.5 Data Types

Pre-defined values or user-entered data are typically stored in main memory. Python manages this through elements called **objects**. Python has **6 basic types** and **5 container types**.

Name	Type	Description
Integers	`int`	Whole numbers, e.g., 3, 200
Floating points	`float`	Numbers with a decimal point: 4.3, 100.0, 236., .21, 2.23e-3
Complex numbers	`complex`	Numbers with real and imaginary parts, e.g., 1 + 2j
Booleans	`bool`	Logical values indicating True or False
Strings	`str`	Ordered sequences of characters: ”Hello”, ’Desmond’, ”2000”
None	`None`	Represents the absence of a value or a null value
Lists	`list`	Ordered sequences of objects: [10, ”Hello”, 200.3]
Tuples	`tuple`	Ordered immutable sequences of objects: (”a”, ”b”)
Dictionaries	`dict`	Key:Value pairs: {”mykey” : ”value”, ”name” : ”Desmond”}
Sets	`set`	Unordered collections of unique objects: {”a”, ”b”}
Frozen Sets	`frozenset`	Immutable version of set: frozenset([”a”, ”b”, ”c”])

The `type()` function returns the class type of the argument (object) passed to it; it is mostly used for debugging. Its syntax is `type(<object>)`.

2.6 Literals

Literals are values associated with data types used in a program. Python has 6 basic literal types.

1. **Integer literals**: 42, -3, 18, 20493, 0.
2. **Floating point literals**: 7.35, -19.93423, 42.0, 5., .5, 5.67e-2.
3. **Complex literals**: 1 + 2j, 3 + 4j.
4. **Boolean literals**: True, False.

5. **String literals**: 'a', 'a++', 'X', 'Desmond', "Cecia", "Alex".
6. **None literal**: None.

Why j, Not i?

In Python the letter `j` denotes the imaginary unit instead of `i`, to avoid confusion with electrical engineering where `i` commonly represents current.

2.7 Booleans: True and False

Python uses `1` to represent `True` and `0` for `False`. Explicit conversion functions translate between the types: `int()` converts a Boolean to an integer, and `bool()` converts a numeric value to a Boolean. Non-zero values are treated as `True`; zero is treated as `False`.

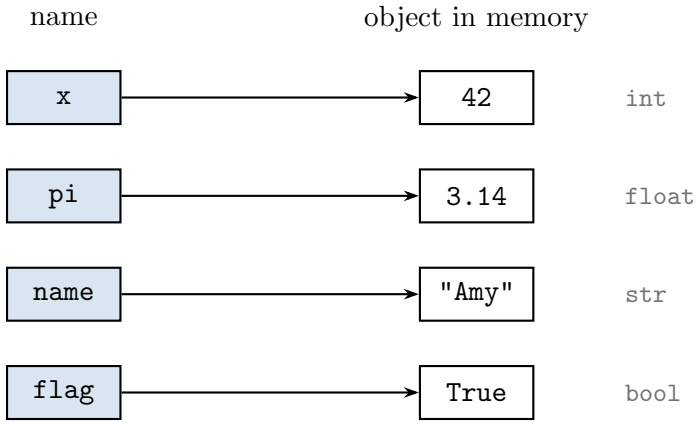
```
print(int(True))     # Print 1
print(int(False))    # Print 0
print(bool(0))       # Print False
print(bool(4))       # Print True
```

2.8 Strings

Strings can be enclosed in single quotes, double quotes, or triple quotes. For this course the convention is: a string with one character uses single quotes (for example `'a'`), and a string with more than one character uses double quotes (for example `"Hello"`).

2.9 Objects, Variables, and References

Data of different types can be stored in the computer's memory. An **object** is a memory location with a type that stores a value – think of it as a box that can hold a single value, or complex data containing multiple values. An object is created the moment a value is first assigned to it, and a **variable** must be defined to reference it.



A variable is a name that refers to an object; type() reports the object's class.

```
<var name> = <initial value>
<var name1> = <var name2> = <initial value>
<var name1>, <var name2>, ... = <initial value1>, <initial value2>,
    ...
```

The name on the left refers to a location in memory and the value on the right is the initially assigned value; this process is called **initialization**.

2.9.1 Rules for Choosing Variable Names

Python imposes the following rules for names (also called **identifiers**, the names that identify variables and functions in a program).

1. Must start with a letter (from the Unicode character set, e.g., a–z, A–Z) or an underscore. It cannot start with a digit.
2. The remaining characters may only be letters, digits (0–9), and underscores.
3. Cannot contain a space or other characters such as a dot, a question mark, an "at" sign, or a hyphen.
4. Has no length limit.
5. Cannot be any of the **reserved words** (keywords), which have special meaning to the Python interpreter.

Reserved keywords include `and`, `as`, `assert`, `async`, `await`, `break`, `class`, `continue`, `def`, `del`, `elif`, `else`, `except`, `False`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `None`, `nonlocal`, `not`, `or`, `pass`, `raise`, `return`, `True`, `try`, `while`, `with`, and `yield`.

Valid examples: `Area`, `Length`, `A`, `seven_And_1`, `my_program`, `desmond`, `GoOd`, `bAd`.

Invalid examples: `my-program` (hyphen), `Python.good` (dot), `7and11` (starts with a digit), `good or bad` (space), `c?0` (question mark), `A/C` (slash), and `pass` (reserved keyword).

2.10 Dynamic Typing and Type Hinting

Python is **dynamically typed**: variables can refer to any type of object. The advantages are flexibility (variables can change type) and ease of use; the disadvantages are that type-related errors may only appear at runtime and that there is less performance optimization than in statically typed languages.

```
x = 10          # x is an integer
x = "Hello"     # x is now a string
```

To improve code quality, maintainability, and collaboration, it is good practice to use **type hinting**, which specifies the type of a variable with a colon followed by the type name.

```
<var name>: <type name> = <initial value>

name: str = "Desmond"
gender: str = 'M'
age: int = 18
height: float = 1.72
is_good_guy: bool = True
```

Hints Are Not Enforced

Type hinting in Python does **not** enforce the type of data assigned. Tools such as `mypy` can analyse the code and check type consistency based on the hints provided.

2.11 Named Constants

A **named constant** (or simply a constant) represents permanent data that never changes. In the `compute_area` example, `PI` is a constant. Python has no special syntax for constants, so a variable named in all uppercase letters (such as `PI`) is used. It remains mutable in principle, but should not be changed.

Benefits: the same value need not be retyped; a change (for example `3.14` to `3.14159`) is made in a single location; and descriptive names make the program easier to read.

Python is **case-sensitive**, so `my_var` and `My_Var` are two different variables. Meaningful names such as `tax` are far better than `t`.

2.12 Explicit Type Conversions

One data type can be converted explicitly into another, for example between floats and integers, or between strings and numbers.

```
val1 = 20
print(val1, type(val1))
print(float(val1), type(float(val1)), '\n')   # 20->20.0 (float)

val2 = 52.7
print(val2, type(val2))
print(int(val2), type(int(val2)), '\n')       # 52.7 -> 52 (int)

val3 = "72"
print(val3, type(val3))
print(int(val3), type(int(val3)), '\n')       # "72" -> 72 (int)

val4 = 86
print(val4, type(val4))
print(str(val4), type(str(val4)))             # 86 -> "86" (str)
```

2.13 Python Convention and Good Programming Style

- **Variable names**: use lowercase letters for a single word; for names of several words, separate them with underscores (e.g., `my_variable`, `total_sum`). This is called **snake case**.
- **Named constants**: use uppercase letters (`PI`) and underscores to separate words (`MAX_LIMIT`).
- **Source filenames**: use lowercase letters and underscores for several words (e.g., `my_program.py`, `data_analysis.py`).

2.14 Mutability of Types

Mutable vs. Immutable

Mutable types can be changed in place: lists (elements added, removed, or modified), dictionaries (key-value pairs added, updated, or removed), and sets (elements added or removed). **Immutable** types create a new object on any change or operation: integers, floats, complex numbers, Booleans, strings, tuples, frozen sets, and `None`. A tuple's elements cannot be changed, but if a tuple contains mutable objects those objects can still be modified.

2.15 Random Number Generation

Python provides the `random` module. Common functions include `random.random()` (a float between 0.0 and 1.0), `random.randint(a, b)` (a random integer between `a` and `b` inclusive), `random.choice(sequence)` (a random element from a non-empty sequence), and `random.sample(population, k)` (a list of `k` unique elements from the population).

Using `random.seed(seed)` initializes the generator so that the same sequence of random numbers is produced every run, which is useful for debugging and reproducibility.

Without a seed, Python uses the current system time or a random source from the operating system, giving a different sequence each run.

```
# Filename: random_number_generation.py
import random

def main():
    random.seed(42)                         # Reproducible seed
    print(random.random())                  # Random float 0.0-1.0
    print(random.randint(1, 10))            # Random int 1 to 10
    print(random.choice(['apple', 'banana', 'cherry']))
    print(random.sample(range(100), 5))  # 5 unique nums 0-99

if __name__ == "__main__":
    main()
```

2.16 Output: The print() Function

Two fundamental elements are outputting data to the standard output device (the screen) and getting user input from the standard input device (the keyboard). The `print()` function outputs data to the screen.

```
print(*<objects>, sep=' ', end='\n', file=sys.stdout, flush=False)
```

`objects` is the value (or values) to be printed; the `*` means any number of arguments may be passed. `sep` is the separator between values (default a space); `end` is printed after all values (default a new line); `file` is the object where values are printed (default `sys.stdout`); and `flush` forces immediate output when set to `True`.

```
# Filename: use_of_end.py
def main():
    course_code = 1023
    print(course_code)
    print("You should be able to see this text", end=' ')
    print("on screen")
    print("How about this text?", end=' ')
    print("Can you see the effect?", end=' ')

if __name__ == "__main__":
    main()

# Filename: use_of_sep_end.py
def main():
    print("Which is the best COMP course? :D")
    print(1, 0, 2, 3)
    print(1, 0, 2, 3, sep='-')          # 1-0-2-3
    print(1, 0, 2, 3, sep='~', end='*') # 1~0~2~3*
    print()                             # Move to next line

if __name__ == "__main__":
    main()
```

2.16.1 Formatted String Literals (f-strings)

An f-string is a string literal prefixed by `f` that may contain placeholders in curly braces; each placeholder holds a Python expression that is evaluated and formatted into the string.

```
print(f"<string> {<expression>}")

# Filename: use_of_fstrings.py
def main():
    x = "A+"; y = 'A'   # ';' joins two statements
    print(f"Desmond will get {x} and Sunny will get {y}")
    print(f"No! Desmond will get {y} and Sunny will get {x}")

if __name__ == "__main__":
    main()
```

2.16.2 Printing Different Types and Multiline Text

```
# Filename: printing_different_types.py
def main():
    name = "Desmond"
    gender = 'M'
    age = 18
    height = 1.72
    print("Hi", name)
    print("Age:", age)
    print("Gender", gender)
    print("Height", height, 'm')

if __name__ == "__main__":
    main()

# Filename: printing_multiline.py
def main():
    print('''COMP 1023 is a Python Course.
I think it is the best COMP course!
''')

if __name__ == "__main__":
    main()
```

A paragraph can be printed by enclosing the text in triple quotes, which preserves the line breaks and formatting. A backslash at the end of a line indicates that a statement continues on the next line, which is useful for long lines broken for readability.

```
# Filename: printing_long_line.py
def main():
    print("COMP 1023 is a Python Course. \
I think it is the best COMP course!")

if __name__ == "__main__":
    main()
```

2.17 Input: The input() Function

User input is read with the `input()` function, whose optional `prompt` parameter is the string displayed on the screen. The entered data is always a **string**.

```
input(<prompt>)

age = input("Enter your age: ")
```

```
print(age)                # Assume the input is 18; prints 18
print(type(age))          # Print <class 'str'>
print(type(int(age)))     # Print <class 'int'>
print(type(float(age)))   # Print <class 'float'>
age = int(age) + 1        # Convert to int and add 1
print("Now you are", age, "years old")
```

2.18 Indentation

Indentation is part of Python's syntax: a difference in indentation disturbs Python's ability to translate the source code into bytecode.

```
print("Welcome to COMP 1023")
  print("Poor indentation")     # IndentationError
```

Indentation Is Syntax

Extra or missing indentation raises an `IndentationError`, because indentation defines the blocks of a program. Keep it consistent throughout the file.

2.19 Good Programming Style

- Put no more than one statement on each line (a semicolon is needed to join two statements).
- Indent statements properly.
- Use blank lines to separate parts of the program.
- Include comments for clarity.
- Use type hinting for better readability.
- Define a `main` function for structured code execution.

Quick Self-Check

Fill in the blanks: comments are added with _____ and _____; _____ is the entry point of a program; objects are referenced using a _____; a named constant uses all _____ letters. (Answers: hash symbols, triple quotes, main(), variable, capital.)

Summary

- Comments are added with `#` (single line) or triple quotes (multi-line); they are not instructions.
- `main()` is the entry point, invoked under the guard `if __name__ == "__main__":`; running a file directly calls it.
- Statements are typical (creation, assignment, calls), branching, or looping.
- Python has 6 basic types (int, float, complex, bool, str, None) and 5 container types (list, tuple, dict, set, frozenset); `type()` reports an object's class.
- Literals are values with a data type; there are 6 basic literal types.
- `True` is 1 and `False` is 0; `int()` and `bool()` convert between them.
- Variables are names referring to objects; names must follow the identifier rules and avoid reserved words.
- Python is dynamically typed; type hinting documents but does not enforce types.
- Named constants use uppercase names; the conventions are snake case, uppercase constants, and lowercase file names.
- Mutable types (list, dict, set) change in place; immutable types (int, float, str, tuple, frozenset, None) do not.
- `random` generates numbers, and `random.seed()` makes results reproducible.

- `print()` writes output (with `sep`, `end`, and f-strings) and `input()` reads a string; indentation is part of the syntax.

3 Python Basic Operations

3.1 Arithmetic Operations

The symbols `+`, `-`, `*`, `/`, `//`, `**` and `%` are called **arithmetic operators**, and the inputs around them are the **arithmetic operands**.

Symbol	Meaning	Example	Result
`+`	Addition	34 + 1	35
`-`	Subtraction	34.0 - 0.1	33.9
`*`	Multiplication	300 * 30	9000
`/`	Float division	1 / 2	0.5
`//`	Integer division	1 // 2	0
`**`	Exponentiation	4 ** 0.5	2.0
`%`	Remainder	20 % 3	2

Underscores in Number Literals

To improve readability, Python allows underscores to separate digits in a number literal, for example `232_45_4519` or `23.24_4545_4519_3415`. The underscore must be placed between two digits, so `45_` or `_45` is incorrect.

3.2 Types of the Results

If one of the operands for `+`, `-`, or `*` is a float, the result is also a float. The same rule applies to `//`, `**`, and `%`. For `/`, the result is **always** a float.

`1 + 2`	`1 + 2.0`	`1.0 + 2`	`1.0 + 2.0`	`1 / 2`	`1 / 2.0`	`1.0 / 2`
3	3.0	3.0	3.0	0.5	0.5	0.5

Implicit Type Conversion

If an integer and a float are involved in a binary operation, Python automatically converts the integer to a float. This is called **implicit type conversion**.

3.3 Integer Division (//) and Exponentiation (**)

The operator `//` performs **integer division** (floor division): it divides two numbers and returns the largest integer that is less than or equal to the result. Examples: `5 // 2` gives 2 (since 2.5 floors to 2); `5.0 // 2` and `5 // 2.0` give `2.0`; `5.5 // 2` gives `2.0`; and `5.5 // -2` gives `-3.0` (the floor of -2.75).

The operator `**` raises a base to a power. Examples: `2 ** 3` is 8; `4.0 ** 0.5` is `2.0`; `2 ** -2` is `0.25`; and `2 ** 3 ** 2` is `2 ** (3 ** 2) = 512` because `**` is right-to-left associative. Tricky: `(-1) ** 0.5` evaluates to `1j`.

3.4 Modulo Operation (%)

The operator `%` is the **modulo** (mod) operator, used to find the remainder after integer division. Useful applications include testing whether a number is even (`number % 2 == 0`) or odd (`number % 2 == 1`), finding the least significant digit (`number % 10`), and finding the last four digits of an HKID (`ID % 10000`).

```
# Filename: mod_operations.py
def main():
    print('1023 % 2 is', 1023 % 2)       # 1
    print('1023 % 10 is', 1023 % 10)     # 3
    print('1023 % 100 is', 1023 % 100)   # 23
    print('1023 % 1000 is', 1023 % 1000) # 23
    print('The last 4 digits of ID:', 8121232 % 10000)  # 1232

if __name__ == "__main__":
    main()
```

The operands may be positive or negative integers or floats. For `a % b` the result is computed by the formula $r = a - (a//b) \times b$. Examples: `10 % 3` is 1; `10 % -3` is -2; `-10 % 3` is 2; `-10 % -3` is -1; `-5.3 % 3` is 0.7; and `5.3 % -3` is -0.7.

Worked problem: display time

Given an amount of time in seconds, print the whole minutes and the remaining seconds. Use integer division for the minutes and the modulo operator for the remainder.

```
# Filename: display_time.py
def main():
    seconds = int(input("Enter an integer for seconds: "))
    minutes = seconds // 60            # whole minutes
    remaining_seconds = seconds % 60   # remaining seconds
    print(seconds, "seconds is", minutes, "minutes and",
   remaining_seconds, "seconds")

if __name__ == "__main__":
    main()
```

For 500 seconds the output is `500 seconds is 8 minutes and 20 seconds`.

3.5 Subtraction and Negation, Division by Zero

The operator `-` is both a binary subtraction operator (two operands, e.g., `10 - 20`) and a unary negation operator (one operand, e.g., `-3`). Division by zero is an error: `5 / 0`, `5 // 0`, and `5 % 0` each raise a `ZeroDivisionError`. Exponentiation is the exception: `0 ** 0` is defined as 1. Always check for zero before dividing.

3.6 Expressions and Precedence

An **expression** is a simple value or a combination of operations that produces a value, such as `89`, `30.23`, or `(3 * 7) + (4 * 4) + 8`. The process of obtaining its value is called **evaluation**. For arithmetic expressions there are three levels of precedence:

1. Exponentiation `**` is applied first.
2. Multiplication `*`, float division `/`, integer division `//`, and remainder `%` are applied next, from left to right.
3. Addition `+` and subtraction `-` are applied last, also from left to right.

Parentheses may be used to override the default precedence.

```
# Filename: precedence_arithmetic_operators.py
def main():
    print("The result of 3 + 4 * 4 + 5 * (4 + 3) - 1 is",
          3 + 4 * 4 + 5 * (4 + 3) - 1)   # 53

if __name__ == "__main__":
    main()
```

Arithmetic operators also apply to variables, with the same precedence. Floating-point operations can show a precision error in their output (for example, `10 - 20.1` prints `-10.100000000000001`).

3.7 Assignment Operations

The assignment operator `=` assigns a value to an object referenced by a variable.

```
<var name> = <initial value>
<var name1> = <var name2> = <initial value>          # cascading
<var name1>, <var name2>, ... = <value1>, <value2>   # parallel
<var name> = <expression>

x, y = 3, 4     # Simultaneous assignments
x = 2 + 9       # x is 11
x = y           # x is 4
x = x + 1       # x is 5
```

`x = x + 1` Is Not an Equation

An assignment such as `x = x + 1` does not follow mathematical rules; in mathematics no real number satisfies $x = x + 1$. In programming it simply assigns the value of `x + 1` to `x`. Using a variable before it is defined, for example `count = count + 1` with no prior definition, raises a `NameError`.

Cascading assignments assign the same value to several variables (`i = j = k = 1` is equivalent to `k = 1; j = k; i = j`). Simultaneous assignment enables swapping: `x, y = y, x` works by tuple packing on the right and tuple unpacking on the left, equivalent to using a temporary variable.

```
# Filename: bmi_example.py
def main():
    weight, height = 195, 70
    bmi = weight / (height * height) * 703
    print("Previous BMI:", bmi)
    weight = 180                     # Update weight
    bmi = weight / (height * height) * 703
    print("Current BMI:", bmi)

if __name__ == "__main__":
    main()
```

3.7.1 Augmented Assignment

Augmented assignment operators simplify coding and are syntactic sugar for the verbose form.

Operator	Description	Example	Equivalent
`+=`	Addition assignment	i += 8	i = i + 8
`-=`	Subtraction assignment	i -= 8	i = i - 8
`*=`	Multiplication assignment	i *= 8	i = i * 8
`/=`	Float division assignment	i /= 8	i = i / 8
`//=`	Integer division assignment	i //= 8	i = i // 8
`%=`	Remainder assignment	i %= 8	i = i % 8
`**=`	Exponentiation assignment	i **= 8	i = i ** 8

For example, `x /= 4 + 5.5 + 1.5` is equivalent to `x = x / (4 + 5.5 + 1.5)`. There are no spaces in these operators: write `+=`, not `+ =`.

3.8 Relational (Comparison) Operations

Relational operators define the relationships between values, and the result is a Boolean value.

Operator	Maths symbol	Description	Example (radius is 5)
`<`	$<$	less than	radius ¡ 0 is False
`<=`	$\leq$	less than or equal to	radius ¡= 0 is False
`>`	$>$	greater than	radius ¿ 0 is True
`>=`	$\geq$	greater than or equal to	radius ¿= 0 is True
`==`	$=$	equal to	radius == 0 is False
`!=`	$\neq$	not equal to	radius != 0 is True

3.8.1 String Comparisons

Strings can be compared with the relational operators using **lexicographical order**, which is based on the ASCII values of characters. For example, `'a'` is 97, `'b'` is 98, `'A'` is 65, and `'B'` is 66. Characters are compared from left to right, and the string with the smaller ASCII value comes first. Thus `"apple" < "banana"` is `True` (since $97 < 98$) and `"Box" < "Boy"` is `True` (since `'x'` 120 $<$ `'y'` 121).

3.9 Logical Operations

There are three logical operators: `not` (negation), `and` (conjunction), and `or` (disjunction). They combine Boolean values and produce a single Boolean result according to the truth table.

p	q	p and q	p or q	not p
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

For example, whether `x` is in the range $[3, 10]$ is `(x >= 3) and (x <= 10)`, which can also be written as the chained comparison `3 <= x <= 10`. Whether $|x| > 1$ is `(x > 1) or (x < -1)`, and `x != 0` is equivalent to `not (x == 0)`.

```
# Filename: leap_year.py
def main():
    year = int(input("Enter a year: "))
    is_leap_year = (year % 4 == 0 and year % 100 != 0) or \
                   (year % 400 == 0)
    print(year, "is a leap year?", is_leap_year)
```

```
if __name__ == "__main__":
    main()
```

A leap year is divisible by 4 but not by 100, or divisible by 400.

3.9.1 Short-Circuiting (Lazy) Evaluation

Short-circuiting halts the evaluation of a Boolean expression as soon as its truth value is determined, which makes computation faster. For `p or q`, `q` is evaluated only if `p` is `False`; for `p and q`, `q` is evaluated only if `p` is `True`. Evaluation proceeds from left to right.

```
print(True or 2/0)    # Evaluates to True, 2/0 is not evaluated
print(False or 2/0)   # Raises an error, since 2/0 is evaluated
print(False and 2/0)  # Evaluates to False, 2/0 is not evaluated
print(True and 2/0)   # Raises an error, since 2/0 is evaluated
```

3.10 Identity and Membership Operations

Identity operators check whether two variables refer to the same object in memory: `is` returns `True` if both operands refer to the same object, and `is not` returns `True` if they do not. Use them to test identity, not equality.

```
a = 1
b = 1
print(a is b)       # Print True
print(a is not b)   # Print False
```

Python caches small integers (typically from -5 to 256), so integers in this range reuse the same object. Integers outside the range are not cached, and `a = 1000; b = 1000` may give `a is b` as `True` in VS Code but `False` in Colab.

Membership operators check whether an item exists within a collection and return a Boolean value: `in` returns `True` if the item exists in a sequence, and `not in` returns `True` if it does not.

3.11 Associativity and Operator Precedence

Associativity is the order in which operators of the same precedence are evaluated. Most arithmetic operators are left-to-right associative, so `10 / 5 * 2` is `(10 / 5) * 2 = 4.0`. Exponentiation `**` is right-to-left associative, so `2 ** 3 ** 2` is `2 ** 9 = 512`.

Operator	Description	Associativity
`**`	Exponentiation	Right-to-left
`+`, `-` (unary)	Unary plus and minus	Right-to-left
`*`, `/`, `//`, `%`	Multiplication, division, integer division, modulus	Left-to-right
`+`, `-` (binary)	Binary addition and subtraction	Left-to-right
`<`, `<=`, `>`, `>=`	Relational operators	Left-to-right
`==`, `!=`	Equality operators	Left-to-right
`is`, `is not`	Identity operators	Left-to-right
`in`, `not in`	Membership operators	Left-to-right
`not`	Logical negation	Left-to-right
`and`	Logical conjunction	Left-to-right
`or`	Logical disjunction	Left-to-right
`=`, `+=`, `-=`, `*=`, ...	Assignment and augmented assignment	Right-to-left

Do Not Rely Only on Precedence

Logical `and` is evaluated before logical `or`, so `(p or q and r)` means `(p or (q and r))`. Use parentheses to make your intention clear rather than relying on precedence.

3.12 Floating-Point Errors

Floating-point numbers are represented with limited precision (typically 15–17 decimal digits), which can cause rounding errors and unexpected results: `0.1 + 0.2` may not equal `0.3`, so `0.1 + 0.2 == 0.3` prints `False`.

```
import math
print(math.isclose(0.1 + 0.2, 0.3))              # Print True

from decimal import Decimal
print(Decimal('0.1') + Decimal('0.2') == Decimal('0.3'))  # True
```

Use `math.isclose()` for comparisons, or the `decimal` module for higher precision.

3.13 Common Pitfalls

- **Assignment vs. equality**: `=` assigns a value, whereas `==` compares two values. Using `=` where `==` is intended is a common mistake.
- **Operator precedence**: in `a + b * c`, multiplication is performed before addition; do not assume operations run left to right.
- `is` **vs.** `==`: `is` checks identity (the same object), while `==` checks equality (the same value). Use `is` only when comparing with `None`.

Summary

- The arithmetic operators `+`, `-`, `*`, `/`, `//`, `**`, and `%` operate on operands; `/` always yields a float and `//` floors the result.
- `**` is right-to-left associative; `%` follows $r = a - (a//b) \times b$ and works with negative operands.
- Division by zero raises `ZeroDivisionError`, but `0 ** 0` is 1.
- Mixed int and float operands trigger implicit type conversion, promoting the int to a float.
- Precedence is `**` first, then `* / // %`, then `+ -`; parentheses override it.
- `=` assigns values; cascading and simultaneous assignments create several objects; augmented operators (`+=`, `-=`, ...) are shorthand.
- Relational operators yield Booleans; strings compare by ASCII lexicographical order.
- `and`, `or`, `not` follow a truth table, with short-circuit evaluation.
- Identity operators (`is`, `is not`) test object identity; membership operators (`in`, `not in`) test existence in a sequence.
- Floating-point errors call for `math.isclose()` or the `decimal` module.

4 Branching (Selection Statements)

Python statements are normally executed **sequentially**, one after another from top to bottom. Sometimes we need the computer to do something only when certain conditions are satisfied; this alters the normal sequential execution and is called **branching** (also **checking and selection**).

- `if` statements: `if`, `if..else`, `if..elif..else`, and **nested** `if`
- `match..case` statements

- Conditional expressions (the **ternary** operator)

4.1 Boolean Expressions and Operators

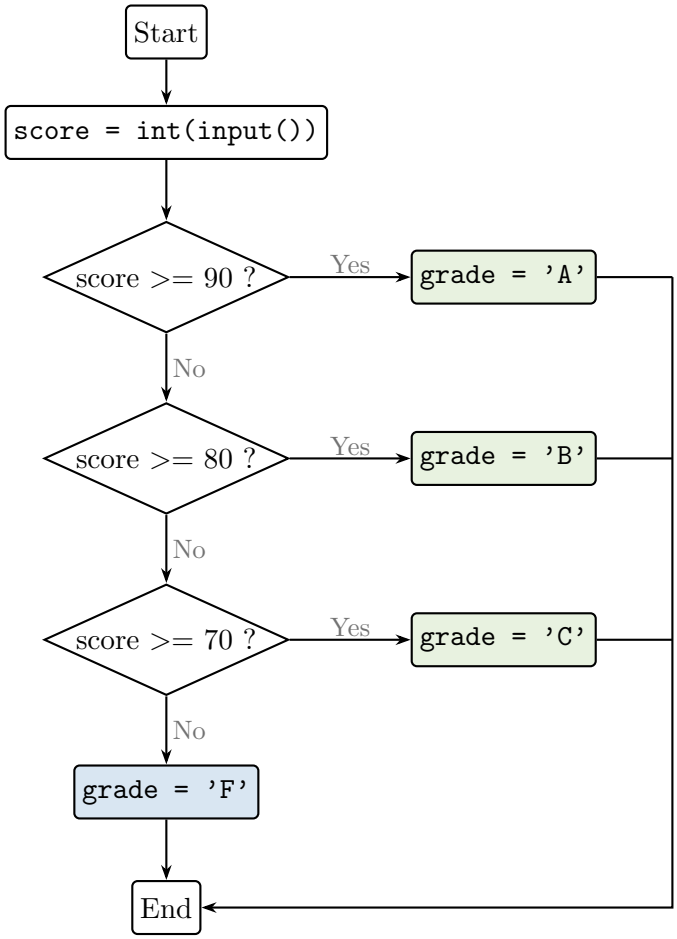
A **Boolean expression** evaluates to either `True` or `False`. Every decision statement makes its choice by evaluating such an expression: based on the `True` or `False` result, the statement takes one of two possible courses. **Relational operators** compare values and produce a Boolean result.

==	!=	<	>	<=	>=
equal	not equal	less than	greater than	at most	at least

Logical operators combine Boolean expressions: `and` is `True` only when both operands are `True`; `or` is `True` when at least one operand is `True`; `not` negates a Boolean value.

4.2 The if Statement

The `if` statement executes a block of statements when a condition is satisfied. Its Boolean expression is tested first; only when it is `True` does control enter the indented body.




```
# Filename: if_mark.py
def main():
    mark = int(input("Enter your mark: "))
    if mark >= 80:
        print("You scored A!")
```

```
    print("Your mark is", mark)

if __name__ == "__main__":
    main()
```

Here `"You scored A!"` is printed only if the input mark is 80 or higher; the final `print` always runs because it is outside the `if` block.

4.3 The if-else Statement

The `if..else` statement executes one block when the condition is satisfied and another block when it is not.

```
# Filename: if_else_temperature.py
def main():
    temperature = float(input("Body temperature: "))
    if temperature > 37.5:
        print("You are having a fever!")
    else:
        print("You have no fever.")

if __name__ == "__main__":
    main()
```

Assigning a value in each branch

Both branches may assign the same variable and only one assignment takes effect, so the variable is always defined after the statement (for example assigning `larger` to `num1` or `num2` depending on which is greater).

4.4 The if-elif-else Statement (if-else Ladder)

When there are more than two cases we use `elif` (short for "else if"). Conditions are checked from top to bottom and only the **first** matching branch runs; the final `else` runs when all previous conditions are `False`. We may have as many `elif` statements as we want.

```
# Filename: if_else_ladder_mark.py
def main():
    mark = int(input("Enter your exam mark: "))
    if mark >= 90:
        grade = 'A'
    elif mark >= 80:
        grade = 'B'
    elif mark >= 70:
        grade = 'C'
    elif mark >= 60:
        grade = 'D'
    else:
        grade = 'F'
    print("You scored", grade, "in your exam.")

if __name__ == "__main__":
    main()
```

A ladder is logically equivalent to a chain of `else: if ... else:` nested blocks, but the `elif` form is far more readable.

4.5 Nested if Statements

Both the `if` branch and the `else` branch may contain `if` statements; the level of nesting can be as large as we want. Finding the maximum of three numbers needs a nested `if`.

```
# Filename: nested_if_max.py
def main():
    num1 = int(input("Enter the 1st number: "))
    num2 = int(input("Enter the 2nd number: "))
    num3 = int(input("Enter the 3rd number: "))
    if num1 >= num2:
        if num1 >= num3:
            maximum = num1
        else:
            maximum = num3
    else:
        if num2 >= num3:
            maximum = num2
        else:
            maximum = num3
    print("The maximum is", maximum)

if __name__ == "__main__":
    main()
```

Indentation decides the block

Indentation is part of the syntax. If a `print` using `area` is placed outside the `if` that assigns `area`, then `area` may never be assigned and the program raises `NameError`. Also, an `else` matches the **nearest** unmatched `if` at the same indentation level: changing indentation changes which condition the `else` belongs to. For example, with `i = 1`, `j = 2`, an `else` aligned with the outer `if i > j` runs (since the condition is false), while the same `else` aligned with an inner `if` does not.

4.6 Truthiness

Any object can serve as a condition. Python treats certain values as **falsy** (equivalent to `False`) and all others as **truthy** (equivalent to `True`). The falsy values are `False`, `None`, `0`, `0.0`, `""`, `[]`, `()` and {}; every other object is truthy.

Truthiness shortens conditions

Instead of `if len(name) > 0:` you may write `if name:`, because an empty string is falsy and a non-empty string is truthy. Likewise `if items:` tests whether a list is non-empty.

4.7 Conditional Expression (Ternary Operator)

A conditional expression performs a check and produces a value in a single line. The Boolean expression is evaluated first and exactly one of the two expressions is returned: <expr 1> if it is `True`, otherwise <expr 2>.

```
# Filename: conditional_expression_odd_even.py
def main():
    n = 5
    result = "even" if n % 2 == 0 else "odd"
    print("n is an", result, "number")
```

```
if __name__ == "__main__":
    main()
```

This is equivalent to, but more compact than, an `if..else` statement that assigns `result` in each branch.

4.8 match-case Statement

The `match-case` statement performs **structural pattern matching**: it can match data structures by shape and content, not just by value, and is often more readable than long `if-elif-else` chains. The matched expression and case values can be a number, a string, or a Boolean value. Case values must have the same data type as the matched value and are constant expressions (they cannot contain variables, so `1 + x` is not allowed). Multiple values may be combined with the pipe operator `|`. Optionally, `case _` is the default case and must be placed **last**.

```
# Filename: match_case_add_subtract.py
def main():
    choice = input("Your choice (A or S): ")
    num1 = int(input("First integer: "))
    num2 = int(input("Second integer: "))
    match choice:
        case 'A' | 'a':          # multiple patterns
            result = num1 + num2
            print(f"{num1} + {num2} = {result}")
        case 'S' | 's':          # multiple patterns
            result = num1 - num2
            print(f"{num1} - {num2} = {result}")
        case _:
            print("Not a proper choice!")

if __name__ == "__main__":
    main()
```

The wildcard must be last

Placing `case _` at the beginning makes the remaining cases unreachable and raises a syntax error. Always put the default case last so the specific cases are evaluated first. Similarly, using the same value twice does not raise an error: only the first matching case runs and later duplicates are silently ignored.

4.8.1 Type Patterns (Extra)

A case can also match a **type pattern**: `int(i)` matches an integer and binds its value to `i`, `float(f)` matches a float, `str(s)` matches a string, and `bool(b)` matches a Boolean. The default `case _` handles any other type, and `type(x).__name__` gives the type name.

```
# Filename: test_type.py
def test_type(x: any) -> None:
    match x:
        case int(i):
            print(f"This is an integer and the value is {i}.")
        case float(f):
            print(f"This is a float and the value is {f}.")
        case str(s):
            print(f"This is a string and the value is \"{s}\".")
        case bool(b):
```

```
            print(f"This is a Boolean and the value is {b}.")
        case _:
            name = type(x).__name__
            print(f"Not an int, float, str or bool, but {name}.")

if __name__ == "__main__":
    test_type(5)       # integer, value 5
    test_type(5.0)     # float, value 5.0
    test_type("123")   # string, value "123"
    test_type(True)    # Boolean, value True
    test_type(2 + 3j)  # complex, caught by case _
```

Aspect	**if-elif-else**	**match-case**
Introduced	Since early Python	Python 3.10
Readability	Verbose with many conditions	Concise for complex patterns
Default case	`else`	Wildcard `_`
Pattern matching	Simple condition checks	Complex patterns (e.g. sequences)

Worked problem: letter grade with a ladder

A program reads an exam mark and prints `A` for 90 or above, `B` for 80 or above, `C` for 70 or above, `D` for 60 or above, otherwise `F`. Because only the first matching branch runs, the conditions must be ordered from highest to lowest; the full program is the `if_else_ladder_mark.py` example above. Once a branch assigns `grade`, later conditions are not tested.

Quick self-check

Fill in the blanks: selection statements include `if`, `if-else`, `if-elif-else`, the `match-case` statement, and conditional expressions. Which statement matches a value against a case and runs the statements of the matched case? (Answer: `match-case`.)

Summary

- Branching alters sequential execution using Boolean expressions that evaluate to `True` or `False`.
- Relational operators (`==`, `!=`, $<$, $>$, $<=$, $>=$) compare values; logical operators (`and`, `or`, `not`) combine them.
- `if` runs a block when a condition holds; `if..else` gives two exclusive branches.
- `if..elif..else` (the ladder) selects the first matching branch among many cases.
- Nested `if` places `if` statements inside a branch; indentation defines every block and binds each `else` to the nearest unmatched `if`.
- Falsy values (`False`, `None`, `0`, `""`, `[]`) act like `False`; everything else is truthy.
- A conditional (ternary) expression chooses between two values in a single line.
- `match-case` performs pattern matching; `case _` is optional and must be last, and type patterns `int(i)`, `float(f)`, `str(s)`, `bool(b)` match by type.

5 Looping

Many tasks inside a program are repetitive, so programming languages provide constructs that control how many times a statement or a block of statements is executed. These constructs are called **loops** (or **iteration statements**).

```
count = 0
while count < 100:
    print("COMP 1023 is the best COMP course!")
    count += 1
```

5.1 Components of a Loop

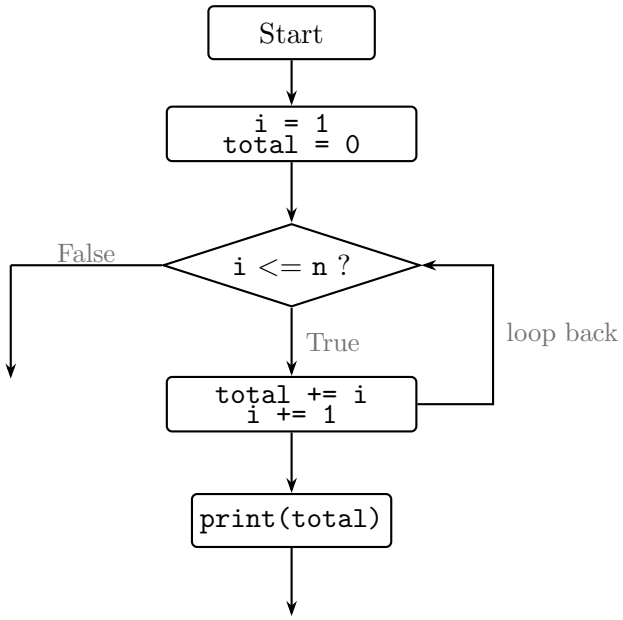
Every loop has four components: **initialize** the loop control variable; **test condition** (a Boolean expression); execute the **loop body** when the condition is `True`; and **update** the control variable, typically through the loop body, before testing again.

Two categories of loops

A **counter-controlled loop** repeats its body a specific number of times that is known before the loop starts. A **sentinel-controlled loop** has an unknown number of repetitions; a **sentinel value** (such as `-1`, different from regular data) decides whether to continue.

5.2 The while Loop

A `while` loop executes a statement or block repeatedly as long as a Boolean expression evaluates to `True`. The condition is tested **before** the body runs.




```
# Filename:  counter_controlled_loop_mark .py
def main():
    total, mark = 0.0, 0.0
    counter = 0                        # initialize
    while counter < 10:                # test condition
        mark = float(input("Enter mark: "))   # loop body
        total += mark
        counter += 1                   # update
    average = total / counter
    print("Average =", average)

if __name__ == "__main__":
    main()
```

Infinite loops

In a counter-controlled loop there must be a point in the loop body that makes the test condition `False`; otherwise the loop runs forever. Here the update `counter += 1` is essential. If it were missing, `counter` would stay 0 and the condition would never become false.

5.2.1 Sentinel-Controlled Loop

The number of iterations is unknown; the loop stops when the user enters the sentinel value `-1`.

```
# Filename: sentinel_controlled_loop_mark .py
def main():
    total, mark = 0.0, 0.0
    counter = 0
    mark = float(input("Enter mark: "))
    while mark != -1:                # test condition
        total += mark                # loop body
        counter += 1
        mark = float(input("Enter mark: "))   # update
    if counter != 0:
        average = total / counter
        print("Average =", average)

if __name__ == "__main__":
    main()
```

Floating-point accumulation

Floating-point numbers are represented by approximation. Summing 0.01, 0.02, ..., 0.99, 1 with `i = i + 0.01` in a `while i <= 1.0` loop yields `49.50000000000003` instead of 50.5, because when the loop ends `i` is slightly larger than 1 and the last value is skipped. Avoid controlling a loop by repeatedly adding a floating-point step when an exact count is needed.

5.3 The for Loop

The `for` statement is used for **counter-controlled** repetition. It iterates over a `range` or over a `sequence` (a collection of items stored in order, such as strings, lists, and tuples). For `range`, the start value is optional (default `0`), the end value is **exclusive**, and the step is optional (default `1`).

```
for <variable> in range(<initial>, <end>, <step>):
    <statement 1>
    ...
```

5.3.1 The range() Function

```
for i in range(5):
    print(i, end=" ")     # 0 1 2 3 4      (1 argument)
for i in range(4, 8):
    print(i, end=" ")     # 4 5 6 7        (2 arguments)
for i in range(3, 9, 2):
    print(i, end=" ")     # 3 5 7          (3 arguments)
```

```
for i in range(5, 1, -1):
    print(i, end=" ")     # 5 4 3 2        (negative step)
```

range() needs integers

The arguments of `range` must be integers. For example `range(1.5, 8.5)` or `range(1.5, 8.5, 1)` is incorrect.

Worked problem: factorial

Compute n! = n * (n-1) * (n-2) * ... * 1 by starting the result at `n` and multiplying by every integer below it:

```
# Filename: factorial.py
def main():
    n = int(input("Enter value of n: "))
    result = n
    for i in range(n - 1, 0, -1):
        result *= i
    print("The factorial of", n, "is", result)

if __name__ == "__main__":
    main()
```

For `n = 5` the program prints `120`; for `n = 12` it prints `479001600`.

5.4 while versus for

A `while` loop is most appropriate for **sentinel-controlled** loops, while a `for` loop is most appropriate for a fixed number of repetitions (**counter-controlled** loops). For the same average computation, the `for` version removes the manual initialize and update bookkeeping: initialize `total = 0.0`, then use `for i in range(10):` to accumulate the marks and divide by 10.

Using a flag variable

Besides a sentinel, a Boolean **flag** controls a loop. Set the flag before the loop (for example `found = False`), change it inside the body when a condition occurs, and test it in the loop condition or afterwards. This is useful when the loop must stop as soon as some event happens.

5.5 The break Statement

The `break` statement alters the flow inside `while` and `for` loops by terminating the **innermost enclosing loop** immediately, before its condition would otherwise become false.

```
# Filename: break_rect.py
def main():
    area = 100.0
    while area > 50.0:
        length = float(input("Enter length of rect: "))
        if length < 5.0:
            break
        width = float(input("Enter width of rect: "))
        if width < 5.0:
            break
```

```
        area = length * width
        print("The area =", area)

if __name__ == "__main__":
    main()
```

The loop terminates if the input length or width is less than 5.0.

5.5.1 Executing a Loop At Least Once

Some languages provide a `do-while` loop that runs the body once before testing the condition, which is useful for repeatedly prompting a user until a valid response is given. Python has no `do-while`, but it can be emulated with `while True` and a strategically placed `break`.

```
# Filename: valid_input.py
def main():
    while True:
        user_input = input("Enter a positive number: ")
        if user_input.isdigit() and int(user_input) > 0:
            break
        print("Invalid input. Please try again.")
    print("Valid input received.")

if __name__ == "__main__":
    main()
```

5.6 The continue Statement

When a `continue` is encountered, control jumps to the beginning of the nearest enclosing loop for the next iteration; all statements after it are skipped for that particular iteration.

```
# Filename: skip_negative_num.py
def main():
    total = 0
    for i in range(8):
        data = int(input("Enter number {}: ".format(i + 1)))
        if data < 0:
            continue
        total += data
    print("The sum =", total)

if __name__ == "__main__":
    main()
```

If the input is negative, control passes to the next iteration and `total += data` is not executed.

5.7 The else Clause of while and for

A `while-else` or `for-else` loop behaves like an ordinary loop, but its extra `else` block runs only when the loop ends **normally** (the condition becomes false, or all iterations are exhausted) without hitting a `break`.

```
while <Boolean expression>:
    <statement A1>
    ...
```

```
else:
    <statement B1>
    ...

# Filename: valid_input_max_attempts.py
def main():
    max_attempts = 3
    attempts = 0
    while attempts < max_attempts:
        user_input = input("Enter a valid number: ")
        if user_input.isdigit():
            print(f"Valid input: {user_input}")
            break
        print("Invalid input.")
        attempts += 1
    else:
        print("Exceeded maximum attempts", max_attempts)

if __name__ == "__main__":
    main()

# Filename: check_prime.py
def main():
    number = int(input("Enter a number: "))
    for i in range(2, number):
        if number % i == 0:
            print(number, "is not a prime number.")
            break
    else:
        print(number, "is a prime number.")

if __name__ == "__main__":
    main()
```

For `number = 13` no divisor is found, so the loop finishes normally and the `else` prints that 13 is prime. For `number = 10`, `break` fires and the `else` is skipped.

else is not "if the loop did nothing"

The `else` block is tied to the absence of `break`, not to whether the body ran. Even a loop that executed its body many times will run `else`, as long as it never hit a `break`.

5.8 Nested Loops

A loop may appear inside another loop; this is a **nested loop**. We can define as many levels as the hardware allows, and different kinds of loops may be nested.

```
# Filename: multiplication_table.py
def main():
    for i in range(1, 10):
        for j in range(1, 10):
            # rjust gives a 3-char right-justified string
            print(str(i * j).rjust(3), end=" ")
        print()

if __name__ == "__main__":
    main()
```

5.9 Variable Scope in Loops

Reusing the loop variable in nested loops

If the same name is used for the loop variable in both an outer and an inner nested loop, the inner loop overwrites the outer value.

```
for i in range(2):
    print("Outer loop: i =", i)
    for i in "OK":        # a string is a sequence
        print("Inner loop: i =", i)
    print("Outer loop after inner: i =", i)
```

After the inner loop, `i` holds the last character `"K"`, so the output shows `i = K` where the outer value was expected. Python allows the same name in nested scopes without creating a new variable; C++ would keep the outer variable separate.

The loop variable also retains its last assigned value after the loop finishes. In the fragment below, `i` is `2` when printed after the loop. Variables defined within loops or branching statements are not limited to that block: they remain accessible in the enclosing function or module scope. In C++, the loop variable is scoped to the loop and cannot be accessed afterwards.

```
for i in range(3):
    print(i)
print(i)   # i will be 2
```

Quick self-check

Fill in the blanks: (1) Python has two repetition statements, the `while` loop and the `for` loop; (2) the part of the loop with the repeated statements is the _____; (3) an _____ loop executes forever; (4) a _____ is a special value that signals the end of input; (5) `break` ends the innermost loop and `continue` ends only the current iteration. Answers: loop body, infinite loop, sentinel value.

Summary

- Loops (**iteration statements**) repeat statements; Python has the `while` and `for` loops.
- Every loop has four parts: initialize, test condition, loop body, and update.
- Loops are counter-controlled (fixed repetitions) or sentinel-controlled (stopped by a sentinel value); a Boolean flag can also control a loop.
- A `while` loop tests its condition before each iteration and is ideal for sentinel-controlled repetition.
- A `for` loop iterates over a `range` or a sequence and is ideal for counter-controlled repetition; `range` accepts 1, 2, or 3 integer arguments.
- `break` terminates the innermost loop immediately; `continue` skips to the next iteration.
- A `while-else` or `for-else` `else` block runs only if the loop ends without a `break`.
- Loops can be nested, and each level may be a different loop type.
- The loop variable persists after the loop and can be overwritten by an inner loop that reuses the same name.

6 Collections: Container Data Types (Part I)

6.1 Collections in Python

Collections are fundamental data structures that store and manage multiple items efficiently. They organise data so it can be accessed, modified, and manipulated easily.

- **Lists**: ordered, mutable collections that can hold mixed data types.
- **Tuples**: ordered, immutable collections typically used to group related data.
- **Sets**: unordered collections of unique elements, useful for membership testing and eliminating duplicates.
- **Dictionaries**: ordered collections (preserving insertion order) of key-value pairs that allow fast lookups.

6.2 Why Lists?

Suppose a program must read 100 numbers, compute their average, and count how many are above the average. Storing each number in its own variable would require writing almost identical code 100 times.

```
def main():
    v1 = float(input("Enter the 1st number: "))
    v2 = float(input("Enter the 2nd number: "))
    ...
    v100 = float(input("Enter the 100th number: "))
    avg = (v1 + v2 + ... + v100) / 100
    print(f"The average is {avg}.")
```

Python instead provides the **list** data type, which stores a sequential collection of elements.

6.3 List Basics

A list is a sequence of any elements of any type.

```
list1 = []                          # Create an empty list
list2 = [2, 3, 4]                   # List with elements 2, 3, 4
list3 = ["red", "green", "blue"] # Create a list with strings
list4 = list(range(3, 6))           # List [3, 4, 5] via list()
list5 = list("abcd")                # List from string "abcd"
list6 = [2, "three", 4]             # List with mixed types

my_list = [5.6, 4.5, 3.3, 13.2, 4.0, 34.33, 34.0, 45.45, 99.993,
   11123]
```

The list `my_list` has 10 elements with indexes from 0 to 9.

Out-of-Bounds Is a Runtime Error

Accessing a list beyond its bounds raises an `IndexError` at runtime. Ensure the index never exceeds `len(my_list) - 1`. The loop below is wrong because its condition is `i <= len(my_list)`.

```
my_list = [5.6, 4.5, 3.3, 13.2, 4.0, 34.33, 34.0, 45.45, 99.993,
   11123]
i = 0
while i <= len(my_list):   # Should be i < len(my_list)
    print(my_list[i])
    i += 1
```

6.4 Operations for Sequences

The following operations are common to sequences such as lists, ranges, tuples, and strings.

Operation	Description
`x in s`	True if element x is in sequence s.
`x not in s`	True if element x is not in sequence s.
`s1 + s2`	Concatenates two sequences s1 and s2.
`s * n, n * s`	n copies of sequence s concatenated.
`s[i]`	ith element in sequence s.
`s[i:j]`	Slice of sequence s from index i to j - 1.
`len(s)`	Length of sequence s (number of elements).
`min(s)`	Smallest element in sequence s.
`max(s)`	Largest element in sequence s.
`sum(s)`	Sum of all numbers in sequence s.
for loop	Traverses elements from left to right.
Comparison	Compares two sequences with `<`, `<=`, `>`, `>=`, `==`, `!=`.

6.4.1 Comparison Operators for Lists

Equality `==` checks whether two lists have the same elements in the same order, and inequality `!=` checks the opposite. The operators `<`, `>`, `<=`, and `>=` compare lists **lexicographically**, element by element until a difference is found.

```
[1, 2, 3] == [1, 2, 3]   # True
[1, 2, 3] == [3, 2, 1]   # False
[1, 2] != [1, 2, 3]      # True
[1, 2, 3] < [1, 2, 4]    # True
[1, 2] < [1, 2, 0]       # True
[1, 2, 3] <= [1, 2, 3]   # True
[1, 2] >= [1, 2, 0]      # False

# Filename: functions_for_lists.py
def main():
    my_list1 = [1, 2, 3, 4, 5]
    my_list2 = [4, 5, 6, 7, 8]
    print("4 in my_list1:", 4 in my_list1)          # True
    print("4 not in my_list1:", 4 not in my_list1)  # False
    print("my_list1 + my_list2:\n", my_list1 + my_list2)
    print("my_list1 * 2:", my_list1 * 2)
    print("2 * my_list1:", 2 * my_list1)
    print("my_list1[3]:", my_list1[3])              # 4
    print("my_list1[3:5]:", my_list1[3:5])          # [4, 5]
    print("len(my_list1):", len(my_list1))          # 5
    print("min(my_list1):", min(my_list1))          # 1
    print("max(my_list1):", max(my_list1))          # 5
    print("sum(my_list1):", sum(my_list1))          # 15
    for i in my_list1:
        print(i, end=" ")
    print()
    print("my_list1 < my_list2:", my_list1 < my_list2)  # True

if __name__ == "__main__":
    main()
```

6.5 The Index Operator

An element in a list is accessed through the index operator, using `my_list[index]`. List indexes are 0-based, ranging from 0 to `len(my_list) - 1`. Because `my_list[index]` can be used like a variable, it is also called an indexed variable.

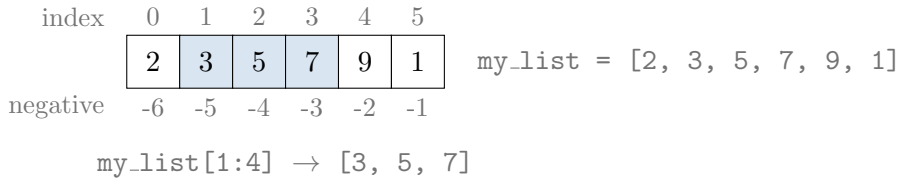
```
my_list[2] = my_list[0] + my_list[1]

for i in range(len(my_list)):
    my_list[i] = i      # assign 0..9 to my_list[0..9]
```

Negative indexes reference positions relative to the end of the list; the actual position is the length added to the negative index. For `my_list = [2, 3, 5, 2, 33, 21]`, `my_list[-1]` is 21 (the same as `my_list[-1 + 6]`) and `my_list[-3]` is 2.

6.6 List Slicing

The slicing operator returns a sublist using `my_list[start : end : step]`. The values `start`, `end`, and `step` must be integers (not floats), and `step` cannot be 0. The slice runs from `start` to `end - 1` with the given step; by default `step` is 1.




```
my_list = [2, 3, 5, 7, 9, 1]
print(my_list[2 : 4])      # Print [5, 7]
print(my_list[0 : 5 : 2])  # Print [2, 5, 9]
print(my_list[1 : -3])     # Print [3, 5]
print(my_list[-4 : -2])    # Print [5, 7]

my_list[1 : 3] = [91, 92, 93, 94]   # replace [3, 5]
print(my_list)             # Print [2, 91, 92, 93, 94, 7, 9, 1]
```

6.6.1 Default Values and Edge Cases

When the start or end index is omitted, defaults are used. For a positive step, the default start is 0 and the default end is `len(my_list)`; if start is greater than or equal to end, the result is an empty list. For a negative step, the default start is the last index and the default end means going until the beginning; if end is greater than or equal to start, the result is an empty list. If start or end is beyond the end of the list, Python uses the length of the list instead.

```
my_list = [2, 3, 5, 7, 9, 1]
print(my_list[ : 2 : 1])   # [2, 3]
print(my_list[3 :   : 1])  # [7, 9, 1]
print(my_list[3 : 1 : 1])  # [] (empty)
print(my_list[ : 2 : -1])  # [1, 9, 7]
print(my_list[3 :   : -1]) # [7, 5, 3, 2]
print(my_list[1 : 3 : -1]) # [] (empty)
print(my_list[3 : 8])      # [7, 9, 1]
print(my_list[7 : 5])      # []
```

Slicing Is Forgiving

Unlike indexing, slicing handles out-of-range indices gracefully: an out-of-range bound is simply replaced by the list length, and an inconsistent range yields an empty list instead of an error.

6.7 Traversing a List

List elements are iterable, so a `for` loop can traverse them without an index variable. An index variable is still needed to traverse in a different order or to change elements.

```
my_list = [5.6, 4.5, 3.3, 13.2, 4.0, 34.33, 34.0, 45.45, 99.993,
   11123]
for u in my_list:
    print(u, end=' ')
# Print 5.6 4.5 3.3 13.2 4.0 34.33 34.0 45.45 99.993 11123

for i in range(0, len(my_list), 2):
    print(my_list[i], end=' ')
# Print 5.6 3.3 4.0 34.0 99.993
```

6.8 List Comprehensions

List comprehensions provide a concise syntax to create a list by processing another sequence. They consist of brackets containing an expression followed by a `for` clause, then zero or more `for` or `if` clauses.

```
list1 = [x for x in range(5)]
print(list1)                    # [0, 1, 2, 3, 4]

list2 = [0.5 * x for x in list1]
print(list2)                    # [0.0, 0.5, 1.0, 1.5, 2.0]

list3 = [x for x in list2 if x < 1.5]
print(list3)                    # [0.0, 0.5, 1.0]
```

6.9 List Methods

Lists are defined by the `list` class; once created, its methods manipulate the list.

Method	Description
`append(x)`	Adds an element x to the end of the list.
`count(x)`	Returns the number of times element x appears.
`extend(anotherList)`	Appends all the elements in anotherList.
`index(x)`	Returns the index of the first occurrence of x.
`insert(index, x)`	Inserts element x at a given index.
`pop(index)`	Removes and returns the element at index (index optional; the last element by default).
`remove(x)`	Removes the first occurrence of element x.
`reverse()`	Reverses the elements in the list.
`sort()`	Sorts the elements in ascending order.

```
list1 = [2, 3, 4, 1, 32, 4]
list1.append(19)
print(list1)              # [2, 3, 4, 1, 32, 4, 19]
```

```
print(list1.count(4))     # 2
list2 = [99, 54]
list1.extend(list2)
print(list1)              # [2, 3, 4, 1, 32, 4, 19, 99, 54]
print(list1.index(4))     # 2
list1.insert(1, 25)
print(list1)              # [2, 25, 3, 4, 1, 32, 4, 19, 99, 54]
print(list1.pop(2))       # 3
print(list1.pop())        # 54
list1.remove(32)          # Equivalent to del list1[4]
list1.reverse()
print(list1)              # [99, 19, 4, 1, 4, 25, 2]
list1.sort()
print(list1)              # [1, 2, 4, 4, 19, 25, 99]
list1.sort(reverse=True)
print(list1)              # [99, 25, 19, 4, 4, 2, 1]
del list1                 # Delete the whole list
```

6.10 Splitting a String into a List

To split the characters in a string `s` into a list, use `list(s)`. The `str` class also contains the `split` method, useful for splitting the items of a string into a list.

```
my_list = list("abc")
print(my_list)   # ['a', 'b', 'c']

items1 = "COMP1023 is the best COMP course".split()   # by spaces
print(items1)  # ['COMP1023', 'is', 'the', 'best', 'COMP', 'course']

items2 = "12/25/2025".split("/")                      # by "/"
print(items2)    # ['12', '25', '2025']
```

6.11 Inputting Lists

Data can be read into a list one item per line in a loop, or entered on a single line separated by spaces.

```
# Filename: inputting_list.py
def main():
    total = 0.0
    counter = 0
    my_list = []
    print("Enter 10 numbers, one number per line: ")
    for i in range(10):
        my_list.append(float(input()))
        total += my_list[-1]
        counter += 1
    average = total / counter
    print("Average =", average)

if __name__ == "__main__":
    main()

# Filename: inputting_list_comprehension.py
def main():
    s = input("Enter 10 numbers separated by spaces: ")
```

```
    items = s.split()                          # Extract items
    my_list = [float(x) for x in items]        # Convert to numbers
    average = sum(my_list) / len(my_list)
    print("Average =", average)

if __name__ == "__main__":
    main()
```

Prefer a List over a Long Ladder

To display a month name from a month number, a long `if-elif-else` ladder is clumsy. A list of month names is clearer: check that the number is between 1 and 12, then print `months[monthNumber - 1]`.

6.12 Copying Lists

The statement `list2 = list1` does **not** copy the contents of the list; it copies the **reference**, so both names refer to the same list. The list previously referenced by `list2` becomes unreferenced and is later reclaimed by the interpreter.

A **shallow copy** creates a new object but inserts references to the nested objects found in the original, so changing a mutable nested object in the copy also affects the original. A **deep copy** recursively copies the nested objects, so changes do not affect the original.

```
# Filename: shallow_vs_deep_copy.py
import copy

original_list = [[1, 2], [3, 4]]

# Shallow Copy -- shared nested objects
shallow_copied_list = original_list.copy()
shallow_copied_list[0][0] = 99        # Also modifies original

# Deep Copy -- independent nested objects
deep_copied_list = copy.deepcopy(original_list)
deep_copied_list[1][0] = 88           # Does not affect original

print(original_list)        # [[99, 2], [3, 4]]
print(shallow_copied_list)  # [[99, 2], [3, 4]]
print(deep_copied_list)     # [[99, 2], [88, 4]]
```

Shallow copies can be made with the list constructor `list(list1)`, a list comprehension `[x for x in list1]`, list concatenation `[] + list1`, or list slicing `list1[:]`. A deep copy uses `copy.deepcopy(list1)`.

6.13 Two-Dimensional Lists

A two-dimensional list consists of rows, where each row is a list of values. Rows are accessed by a row index and values within a row by a column index, so each value is reached with `matrix[i][j]`.

```
matrix = [
    [1, 2, 3, 4, 5],
    [6, 7, 0, 0, 0],
    [0, 1, 0, 0, 0],
    [1, 0, 0, 0, 8],
    [0, 0, 9, 0, 3]
```

```
]
# matrix[0] is [1, 2, 3, 4, 5]; matrix[0][0] is 1; matrix[4][4] is 3

# Filename: summing_elements.py
def main():
    matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

    # Summing All Elements
    total = 0
    for row in matrix:
        for value in row:
            total += value
    print("Total is", total)          # 45

    # Summing Elements by Column
    for column in range(len(matrix[0])):
        total = 0
        for row in range(len(matrix)):
            total += matrix[row][column]
        print("Sum for column", column, "is", total)

if __name__ == "__main__":
    main()
```

A two-dimensional list can be printed either by using indices or by direct iteration, printing a new line after each row. Higher dimensions are also possible: a three-dimensional list can store, for example, the exam scores of 6 students with 5 exams each having 2 parts, where `scores[0][1][0]` refers to the multiple-choice score of the first student's second exam.

Summary

- A **list** stores a sequential collection of elements of any type; it is ordered and mutable.
- Sequence operations include `in`/`not in`, concatenation `+`, repetition `*`, indexing, slicing, `len`/`min`/`max`/`sum`, traversal, and comparison.
- Indexing is 0-based and supports negative indexes; an out-of-range index raises `IndexError`.
- Slicing `s[i:j:k]` takes a sublist, with sensible defaults and gracefully handled out-of-range bounds.
- List comprehensions build a list from another sequence concisely.
- The `list` methods `append`, `count`, `extend`, `index`, `insert`, `pop`, `remove`, `reverse`, and `sort` manipulate a list.
- `list(s)` and `s.split()` turn a string into a list.
- `list2 = list1` copies a reference, not the list; use a shallow copy (constructor, comprehension, `+`, slicing) or `copy.deepcopy` for a deep copy.
- A two-dimensional list is a list of rows accessed with `matrix[i][j]`.

7 Functions (Part 1)

7.1 Why Functions?

Motivating Problem

Suppose we need the sum of the integers from 1 to 10, from 20 to 37, and from 35 to 49. Without functions, the same loop is repeated three times, changing only the range of values.

```
# Repeated, nearly identical code
total = 0
for i in range(1, 11):
    total += i
print("Sum of integers from 1 to 10 is", total)
# ... the same loop again for 20..37 and for 35..49
```

In Python we do this by defining a **function**:

```
# Filename: sum_of_integers_func.py
def sum_range(i1, i2):
    result = 0
    for i in range(i1, i2 + 1):
        result += i
    return result

def main():
    print("Sum of 1 to 10 is", sum_range(1, 10))
    print("Sum of 20 to 37 is", sum_range(20, 37))
    print("Sum of 35 to 49 is", sum_range(35, 49))
```

The Machine Model

Think of a function as a machine: **inputs** (the arguments) are the raw materials, **processing** is the work the machine performs, and the **output** is the finished product it returns. For example, `add(5, 3)` feeds 5 and 3 in and produces 8.

7.2 Defining a Function

A **function** is a collection of statements grouped together that performs an operation, such as `input("Enter a value ")`.

Function Syntax

```
def <function_name>(<parameter_list>):
    <statement_1>
    ...
    <statement_N>
```

`function_name` is the function's name, `parameter_list` is a list of parameters (variables), and the statements are executed when the function is called (invoked).

Note that `main` is itself a function, serving as the entry point of a program.

```
# Filename: maximum_two_numbers.py
def max(num1, num2):
    """Return the larger of the two given numbers."""
    if num1 > num2:
```

```
        result = num1
    else:
        result = num2
    return result

def main():
    x, y = 10, 20
    z = max(x, y)   # Call the function
    print("The larger number is", z)
```

Terminologies

- **Function name**: here `max`.
- **Formal parameters**: `num1` and `num2`; the **parameter list** is `num1, num2`.
- **Function header**: `def max(num1, num2)`; the **body** implements the logic.
- **Return value**: produced by the return statement.
- **Function caller**: `main()`.
- **Actual parameters** (arguments): here `x` and `y`.

A **docstring** is an optional string literal placed right after the header to document the function.

Function Order Does Not Matter

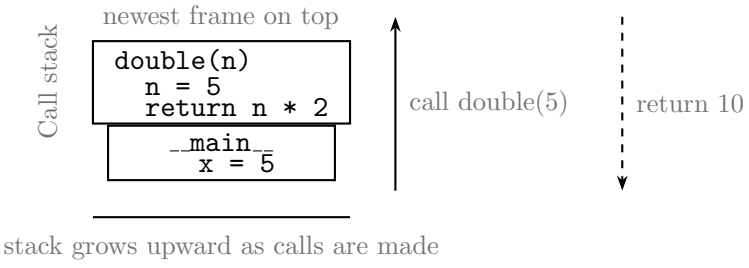
A function may be defined in any order inside a `.py` file, because a function is loaded into memory when it is called. So `main` may be defined before or after `max`.

7.3 The Flow of a Function Call

When a function is called, control is transferred to it; when the function finishes, control returns to the point where it was called.

Activation Records and the Call Stack

Activation records (also called **stack frames**) store information about active functions: their parameters, local variables, and return addresses. Each time a function is called, a new activation record is created on the **call stack**.



Parameter Passing by Reference

`x` references an integer object with value 10. Passing `x` to `num1` passes the reference of that object, so `x` and `num1` point to the same object while the value does not change. The same holds for `y` and `num2`.

7.4 Type Hints in Functions

Type hinting lets you specify the expected types of parameters and of return values. It improves readability and helps with static type checking.

Type Hinting Syntax

Use a colon (`:`) to annotate a parameter type and an arrow (`->`) to annotate the return type. Benefits include clearer documentation and easier debugging and maintenance.

```
def max(num1: int, num2: int) -> int:
    if num1 > num2:
        result: int = num1
    else:
        result: int = num2
    return result
```

Hints Are Not Enforcement

Type hints are only annotations. Python does not stop you from passing a value of the wrong type; the hints help readers and static checkers.

7.5 Functions That Return `None`

Technically every function returns a value. If a function does not return a value, by default it returns the special value `None`. Such a function is called a **None function** or **void function**.

Returning `None`

A return statement is not needed for a None (void) function, but it may be used to terminate the function and return control to the caller. The syntax is `return` or `return None`.

```
def sum(number1: int, number2: int) -> None:
    total: int = number1 + number2
    # No return statement

def main() -> None:
    print(sum(1, 2))    # Print None
```

7.6 Positional and Keyword Arguments

There are two kinds of arguments passed to the formal parameters.

Positional vs. Keyword Arguments

- **Positional arguments** must be passed in the same order as their parameters in the function header.
- **Keyword arguments** are passed in the form `name = value`, so their order does not matter.

```
# Filename: print_n_times.py
def print_n_times(text: str, n: int) -> None:
    for i in range(n):
        print(text)

def main() -> None:
    print_n_times("COMP 1023", 3)          # Positional
    print_n_times(n=3, text="COMP 1023")   # Keyword
```

Positional After Keyword

You may mix positional and keyword arguments, but a positional argument cannot appear after any keyword argument.

```
def sum(value1: int, value2: int, value3: int) -> int:
    return value1 + value2 + value3

def main() -> None:
    print(sum(10, value2=20, value3=30))  # Print 60
    # print(sum(10, value2=20, 30))  # Error: pos after keyword
```

7.7 Default Parameter Values

Python allows **default argument values**, used when a function is called without the corresponding arguments.

```
# Filename: print_area.py
def print_area(width: float = 1, height: float = 2) -> None:
    area: float = width * height
    print("Width:", width, "\theight:", height, "\tarea:", area)

def main() -> None:
    print_area()                    # width = 1, height = 2
    print_area(4, 2.5)              # width = 4, height = 2.5
    print_area(height=5, width=3) # Keyword arguments
    print_area(10)                  # width = 10, height = 2
    print_area(width=1.2)           # height uses default 2
```

Rule for Default Arguments

Default arguments must be placed after required arguments.

Default Before Required Is a Syntax Error

Writing a default `height` before the required `width` raises a syntax error, because a default argument cannot come before a required one.

7.8 Returning Multiple Values

A function may return multiple values. The values are packed into a **tuple**, which can be unpacked directly into several variables.

```
# Filename: sort_numbers.py
def sort(number1: int, number2: int) -> tuple[int, int]:
    if number1 < number2:
        return number1, number2
    else:
        return number2, number1

def main() -> None:
    n1: int; n2: int
    n1, n2 = sort(3, 2)     # n1 is 2, n2 is 3
    print("n1 is", n1)
    print("n2 is", n2)
```

7.9 The pass Statement

The `pass` statement is a null operation: when it runs, nothing happens. It is a placeholder where syntactically some code is required but no action is needed.

Uses of pass

- Empty function bodies and empty classes.
- Selection/branching statements.
- Looping/iterative statements.

```
def my_function() -> None:
    pass    # Empty function body

x: int = 5
if x > 10:
    pass    # No action if x > 10
else:
    print("x is not greater than 10")
```

Quick Self-Check

1. A function header begins with which keyword, followed by the function's name and its parameter list, and ends with a colon?
2. What is a function called if it does not return a value?
3. What are the two kinds of arguments, and in what order may they be mixed?

Summary

- A **function** groups reusable statements; define it with `def`, a header, an indented body, and optionally a docstring.
- Calling a function transfers control to it and back; each call creates a new **activation record** (stack frame) on the call stack.
- **Type hints** annotate parameter types with `:` and return types with `->`, but are not enforced at run time.
- A function with no return value returns `None` (a **None**/**void** function).
- Arguments may be **positional** or **keyword**; positional arguments cannot follow keyword arguments.
- **Default arguments** must come after required arguments.
- Functions may return **multiple values** packed into a tuple.
- `pass` is a null placeholder for empty bodies or blocks.

8 Functions (Part 2)

8.1 Scope of Variables

The **scope** of a variable is the part of the program where the variable can be referenced.

Local vs. Global Variables

- A variable created inside a function is a **local variable**: it can only be accessed within the function, and its scope starts from its creation and continues to the end of the function.
- A variable created outside all functions is a **global variable**: it is accessible to all functions in its scope.

```
# Filename: scope_example.py
x: int = 1          # Global variable
def f() -> None:
    y: int = 2      # Local variable
    print(x)        # Print 1
    print(y)        # Print 2

f()
print(x)            # Print 1
# print(y)          # NameError: name 'y' is not defined
```

Variables Created Conditionally

A variable is only created when its assignment runs. If the user enters a non-positive value below, line 4 raises a `NameError` because `y` is never created.

```
x: int = int(input("Enter an integer: "))
if x > 0:
    y: int = 4
print(y)    # Error when x is not positive
```

The LEGB Rule

When a name is used, Python searches **L**ocal, then **E**nclosing (any enclosing function), then **G**lobal (module level), and finally **B**uilt-in (e.g. `print`). The first match wins.

8.2 The `global` Statement

We can bind a local variable to the global scope, or create a variable inside a function and use it outside, with the `global` statement.

```
# Filename: global_example1.py
def create_global() -> None:
    global x
    x = 7
    print(x)    # Print 7

create_global()
print(x)        # Print 7

# Filename: global_example3.py
x: int = 20     # x is a global variable

def modify_global() -> None:
    global x    # Commenting out this line causes an error
    x = x + 30  # Modifying the global variable

modify_global()
print(x)        # Print 50
```

`UnboundLocalError`

Without `global x`, the statement `x = x + 30` is treated as assigning a local variable `x`. Since no local `x` has a value yet, Python raises an `UnboundLocalError` when evaluating `x + 30`.

8.3 Name Conflicts

```
# Filename: name_conflict.py
x: int = 1              # Global variable

def f() -> None:
    x: int = 2          # Local variable shadows the global
    print(x)            # Print 2

f()
print(x)                # Print 1
```

A local variable with the same name shadows the global one inside the function. From the point of the local assignment on, the global `x` is not accessible within the function; outside the function it is still accessible, so the last line prints 1.

8.4 Memory: Stack Frame, Global Frame, and Heap

Memory Regions

- **Stack frame**: a block of memory storing information about a function call. A new stack frame holds its local variables, parameters, and return address when the function is called, and is removed when the function finishes.
- **Global frame**: the top-level scope of the program, containing global variables and functions accessible from anywhere. It is created when the interpreter starts.
- **Heap**: a region for dynamic allocation, where objects (integers, floats, lists, dictionaries, classes) are stored when created. It is managed automatically, and objects can outlive the functions that created them.

8.5 Modularizing Code

We can place function definitions into a file called a **module** with the `.py` extension. The module can later be imported into a program for reuse; it should sit in the same directory as the other programs and may contain more than one function.

Two Import Styles

- `from gcd_function import gcd` imports the name directly; call it as `gcd(n1, n2)`.
- `import gcd_function` imports the module; call the function as `gcd_function.gcd(n1, n2)`.

```
# Filename: gcd_function.py
def gcd(n1: int, n2: int) -> int:
    result: int = 1     # Initial gcd is 1
    k: int = 2          # Possible gcd is 2
    while k <= n1 and k <= n2:
        if n1 % k == 0 and n2 % k == 0:
            result = k  # Update gcd
        k += 1
    return result
```

Redefining a Function

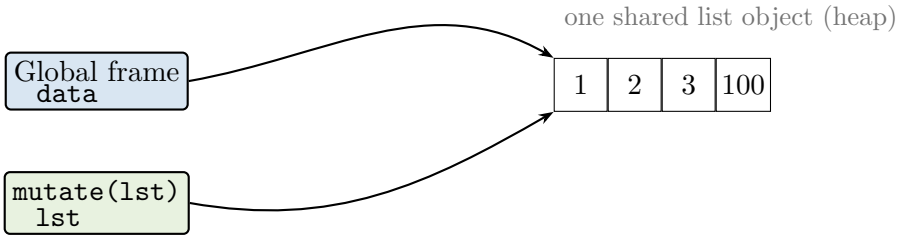
What happens if we define two functions with the same name in a module? There is no syntax error, but the latter definition prevails.

8.6 Passing Lists to Functions

When passing a list to a function, the contents of the list may change after the call, since a list is a **mutable object**.

```
# Filename: passing_list_to_func1.py
def m(number: int, numbers: list[int]) -> None:
    number = 1001       # Rebind the local name
    numbers[0] = 5555   # Modify the shared list

def main() -> None:
    x: int = 1
    y: list[int] = [1, 2, 3]
    m(x, y)
    print("x is", x)          # Print x is 1
    print("y[0] is", y[0])    # Print y[0] is 5555
```



Both names refer to the SAME object, so an in-place change inside `mutate()` is visible to the caller.

Mutable Default Argument Trap

When a default parameter is a mutable type (e.g. a list), the default object is created once when the function is defined, not on each call, so changes made during one call persist in later calls.

```
# Filename: passing_list_to_func2.py
def add(x: int, l: list[int] = []) -> list[int]:
    if x not in l:
        l.append(x)
    return l

def main() -> None:
    print(add(1))             # Print [1]
    print(add(2))             # Print [1, 2], not [2]
    print(add(3, [11, 12]))   # Print [11, 12, 3]
```

The Safe Pattern: Use `None`

Set the default value to `None` and create a fresh list inside the function, so each call gets its own list.

```
# Filename: passing_list_to_func2_fix.py
def add(x: int, l: list[int] | None = None) -> list[int]:
    if l is None:
        l = []
    if x not in l:
        l.append(x)
    return l
```

Returning a List

When a function returns a list, the list's reference is returned, so the caller and the function share the same object.

8.7 Variadic Arguments: `*args` and `**kwargs`

Python allows functions to accept any number of parameters, which is handy for a varying amount of input data.

`*args` and `**kwargs`

- `*args` collects a variable number of non-keyword arguments into a **tuple**.
- `**kwargs` collects a variable number of keyword arguments into a **dictionary**.

```
# Filename: example_any_num_parameters.py
def example_function(*args: int, **kwargs: str | int) -> None:
    print("Arguments:", args, type(args))
    print("Keyword Arguments:", kwargs, type(kwargs))

def main() -> None:
    example_function(1, 2, 3, name='Alice', age=30)
# Arguments: (1, 2, 3) <class 'tuple'>
# Keyword Arguments: {'name': 'Alice', 'age': 30} <class 'dict'>
```

8.8 Lambda Functions

Lambda functions are anonymous expressions: they have no name unless explicitly assigned to a variable. A lambda can take any number of arguments but can only contain one expression.

Lambda Syntax

```
lambda <arguments> : <expression>
```

```
from collections.abc import Callable
result: int = (lambda x, y: x + y)(1, 2)   # Return 3
func: Callable[[int, int], int] = lambda x, y: x + y
print(func(1, 2))                           # Print 3

greet: Callable[[], str] = lambda: "Hello!"  # No parameters
print(greet())
```

When to Use Lambda

Lambdas are not inherently faster than standard functions, since both compile to similar bytecode. They reduce boilerplate when passed directly to higher-order functions such as `map` or `filter`, avoiding a separate named function.

8.9 Common Use Cases: `filter`, `map`, and `reduce`

Lambdas are often used in functional programming with `filter()`, `map()`, and `reduce()`, which take other functions as arguments to process a collection.

`filter()`, `map()`, and `reduce()`

Each takes a function and an iterable.
- `filter()`: keeps each item for which the function returns `True`, discarding the rest.
- `map()`: keeps the function's returned value for each item.
- `reduce()`: applies the function to the first two items, then to the result and the next item, and so on; import it with `from functools import reduce`.

```
from functools import reduce
numbers: list[int] = [1, 2, 3, 4, 5, 6, 7, 8]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)        # Print [2, 4, 6, 8]

fruits: list[str] = ['apple', 'banana', 'cherry']
lengths = list(map(lambda x: len(x), fruits))
print(lengths)      # Print [5, 6, 6]

result: int = reduce(lambda x, y: x + y, numbers)
print(result)       # Print 36
```

Quick Self-Check

1. A variable created in a function is called what, and when does its scope begin and end?
2. Which type of variable is created outside all functions and is accessible to all functions in its scope?
3. Which standard-library module provides `reduce`?

Summary

- **Local variables** live inside a function; **global variables** live at module level and follow the **LEGB** lookup order.
- Use the `global` statement to rebind a global name inside a function; otherwise assignment creates a local and may raise `UnboundLocalError`.
- Memory is organized as the **stack frame** (per call), the **global frame** (module scope), and the **heap** (objects).
- A **module** is a `.py` file of reusable functions, imported with `import` or `from ... import ...`.
- Lists are **mutable**, so passing or returning them shares references; a **mutable default argument** is created only once, so prefer `None` as the default.
- `*args` collects extra positional arguments into a tuple and `**kwargs` collects extra keyword arguments into a dictionary.
- **Lambda** functions are anonymous single-expression functions, often combined with `filter`, `map`, and `reduce` (the latter from `functools`).

9 Collections: Container Data Types (Part II)

9.1 Tuples

Tuples are similar to lists, but their elements are fixed: once a tuple is created, its elements cannot be added, deleted, replaced, or reordered. If the contents of a list should not change, a tuple prevents accidental modification. Tuples are generally more efficient than lists due to Python's internal optimizations. A tuple is created by enclosing its elements in parentheses, separated by commas.

```
tuple1: tuple[()] = ()  # Create an empty tuple
```

```
tuple2: tuple[int, ...] = (1, 3, 5)  # Create (1, 3, 5)
tuple3: tuple[str, ...] = ("red", "green", "blue")  # 3 strings
tuple4: tuple[int, ...] = tuple([1, 2, 3])  # From a list
tuple5: tuple[int, ...] = tuple([2 * x for x in range(1, 5)])
tuple6: tuple[str, ...] = tuple("abcd")  # From a string
tuple7: tuple[int, str, int] = (1, "two", 3)  # Mixed types

my_tuple: tuple[float, ...] = (5.6, 4.5, 3.3, 13.2, 4.0, 34.33, 34.0,
   45.45, 99.993, 11123)
```

The tuple `my_tuple` has 10 elements with indexes ranging from 0 to 9.

Out-of-Bounds Is a Runtime Error

As with lists, accessing a tuple beyond its bounds raises an `IndexError`. Ensure the index never exceeds `len(my_tuple) - 1`; a loop condition of `i <= len(my_tuple)` is wrong and should be `i < len(my_tuple)`.

9.1.1 Functions for Tuples

```
# Filename: functions_for_tuples.py
def main() -> None:
    my_tuple1: tuple[int, ...] = (1, 2, 3, 4, 5)
    my_tuple2: tuple[int, ...] = tuple([4, 5, 6, 7, 8])
    print("4 in my_tuple1:", 4 in my_tuple1)              # True
    print("4 not in my_tuple1:", 4 not in my_tuple1)      # False
    print("my_tuple1 + my_tuple2:\n", my_tuple1 + my_tuple2)
    print("2 * my_tuple1:", 2 * my_tuple1)
    print("my_tuple1[3]:", my_tuple1[3])                  # 4
    print("my_tuple1[3:5]:", my_tuple1[3:5])              # (4, 5)
    print("my_tuple1[-1]:", my_tuple1[-1])                # 5
    print("len(my_tuple1):", len(my_tuple1))              # 5
    print("min(my_tuple1):", min(my_tuple1))              # 1
    print("max(my_tuple1):", max(my_tuple1))              # 5
    print("sum(my_tuple1):", sum(my_tuple1))              # 15
    for i in my_tuple1:
        print(i, end=" ")
    print()
    print("my_tuple1 < my_tuple2:", my_tuple1 < my_tuple2)  # True
    del my_tuple1          # Delete the whole tuple
if __name__ == "__main__":
    main()
```

Comparison operators work exactly as for lists: `==` and `!=` test equality, and `<`, `>`, `<=`, `>=` compare tuples **lexicographically**, element by element until a difference is found.

9.1.2 Indexing, Immutability, and Mutable Elements

An element is accessed with `my_tuple[index]`, and indexes are 0-based. Because tuples are immutable, assigning to an element is an error: `my_tuple[1] = 10` is not allowed.

Immutable Tuple, Mutable Contents

Although a tuple's elements cannot be added, deleted, or replaced, a tuple may **contain** mutable objects (such as lists or class instances) whose internal state can still change. For example, a tuple of `Circle` objects cannot have its elements replaced, but the radius of an individual circle can be changed.

Negative indexes reference positions relative to the end: for `my_tuple = (2, 3, 5, 2, 33, 21)`, `my_tuple[-1]` is 21 and `my_tuple[-3]` is 2 – exactly the same as lists.

9.1.3 Tuple Slicing

Slicing works like lists: `my_tuple[start : end : step]` returns a sub-tuple from `start` to `end - 1`. The defaults and edge-case behaviour are identical to lists, but a slice of a tuple cannot be assigned a new value.

```
my_tuple: tuple[int, ...] = (2, 3, 5, 7, 9, 1)
print(my_tuple[2:4])      # (5, 7)
print(my_tuple[0:5:2])    # (2, 5, 9)
print(my_tuple[1 : -3])   # (3, 5)
print(my_tuple[-4 : -2]) # (5, 7)
# my_tuple[1 : 3] = (91, 92, 93, 94)   # Error! Tuples are immutable
```

9.1.4 Traversing, Comprehensions, and Methods

Tuple elements are iterable and can be traversed with a `for` loop, exactly like lists, using an index variable when a different order is needed. Python does not support tuple comprehensions directly; instead use the `tuple()` function with a generator expression: `tuple(x for x in range(5))`.

Method	Description
`count(element)`	Returns the number of times the given element appears.
`index(element, start, end)`	Returns the first occurrence of the element, searching from start to end.

```
my_tuple1: tuple[int, ...] = (0, 1, 2, 3, 2, 3, 1, 2, 3)
c1: int = my_tuple1.count(3)     # 3
pos1: int = my_tuple1.index(3)   # 3
pos2: int = my_tuple1.index(3, 4)  # 5
# pos3 = my_tuple1.index(4)      # Error: 4 is not in the tuple
```

To split a string into a tuple, use `tuple(s)`; a string can also be split into a list with `split` and then converted to a tuple.

9.1.5 Copying Tuples, Two-Dimensional Tuples, and Packing

The statement `tuple2 = tuple1` copies the reference, not the contents: both names then refer to the same tuple, and the tuple previously referenced by `tuple2` is reclaimed. Because tuples are immutable, a deep copy is usually unnecessary; however, if a tuple contains mutable objects (such as lists), `copy.deepcopy()` can create a new tuple with deep copies of them.

A **two-dimensional tuple** consists of rows, each row being a tuple, accessed with `matrix[i][j]`. Higher dimensions are also possible, for example a three-dimensional tuple of exam scores where `scores[0][1][0]` is the multiple-choice score of the first student's second exam.

```
t: tuple[int, ...] = (4, 5, 1)   # automatic packing of a tuple
v1, v2 = range(2, 4)             # unpacking: v1 is 2 and v2 is 3
```

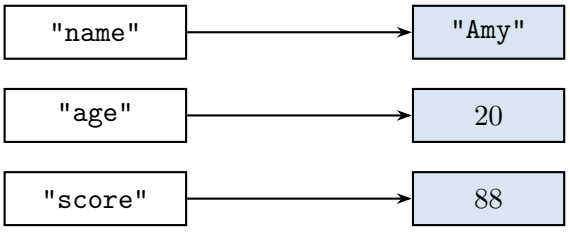
Automatic Packing and Unpacking

A tuple can be created from comma-separated values (automatic packing), and a sequence can be unpacked into several variables at once. The statement `return v1, v2` actually returns a tuple, which underlies simultaneous assignment and swapping.

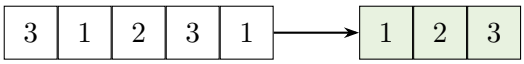
9.2 Dictionaries

A **dictionary** is a container object that stores a collection of **key/value pairs**. It enables fast retrieval, deletion, and updating of values using keys. A dictionary cannot contain duplicate keys: each key maps to exactly one value, and the pair forms an item (or entry). It is also known as a **map**, and resembles a word dictionary where words are keys and definitions are values.

dict : key → value



set : unique, unordered → duplicates removed




```
my_dict1: dict[str, any] = {}           # Create an empty dictionary
my_dict2: dict[str, any] = dict()       # Create an empty dictionary

my_dict3: dict[str, str] = {  # Create a dictionary with two items
    "21053124": "Tammy",
    "21543257": "Elvis"
}

my_dict4: dict[str, str] = dict(name="Tammy", id="Elvis")

my_dict5: dict[str, str] = dict([  # From list of tuples
    ("21053124", "Tammy"),
    ("21543257", "Elvis")
])

my_dict6: dict[int, int] = {x: x ** 2 for x in range(5)}  # comp
```

Keys are not limited to strings, but they must be of a **hashable** type such as numbers and strings.

```
# Filename: functions_for_dictionary.py
def main() -> None:
    students1: dict[str, str] = { "21053124": "Tammy", "21543257":
   "Elvis" }
    students2: dict[str, str] = { "22356267": "Peter", "25141321":
   "John" }
```

```
    print("21053124 in students1:", "21053124" in students1)  # True
    print("21053124 not in students1:", "21053124" not in students1)
   # False
    # print(students1 + students2)  # Error: + not supported
    # print(2 * students1)  # Error: * not supported
    print("len(students1):", len(students1))  # 2
    print("min student ID:", min(students1))  # 21053124
    print("max student ID:", max(students1))  # 21543257
    for key in students1:
        print(key + ": " + str(students1[key]))
    print("students1 == students2:", students1 == students2) # False
    print("students1 != students2:", students1 != students2) # True
if __name__ == "__main__":
    main()
```

9.2.1 Adding, Modifying, Retrieving, and Deleting Items

- `dictionaryName[key] = value` adds an item; if the key already exists, its value is replaced.
- `dictionaryName[key]` retrieves a value; if the key is absent, an error occurs.
- `del dictionaryName[key]` deletes the item; if the key is absent, an error occurs.

```
def main() -> None:
    students: dict[str, str] = { "21053124": "Tammy", "21543257":
   "Elvis" }
    students["27272312"] = "Desmond"       # Add a new item
    print(students["27272312"])            # Desmond
    students["21053124"] = "Tammy Wong"  # Replace the value
    print(students["21053124"])            # Tammy Wong
    del students["27272312"]  # Delete item with key "27272312"
    # print(students["22222222"])          # Raises KeyError

if __name__ == "__main__":
    main()
```

No Subscript Indices or Slicing

Dictionaries are accessed by keys, not positions, so `[0]`, `[1]`, ... and slicing such as `dict[1:3]` do not work – unless the integers happen to be actual keys.

9.2.2 Traversing a Dictionary

```
def main() -> None:
    students: dict[str, str] = { "21053124": "Tammy", "21543257":
   "Elvis" }
    for key in students:  # Access values by key
        print(key + ": " + str(students[key]))
    for key in students.keys():  # Iterate through keys
        print(key)
    for value in students.values():  # Iterate through values
        print(value)
    for key, value in students.items():  # key-value pairs
        print(str(key) + ": " + str(value))

if __name__ == "__main__":
    main()
```

Do Not Modify While Traversing

Do not modify a dictionary while traversing it (without making a copy). Python raises a `RuntimeError` if the size of the dictionary changes during iteration.

9.2.3 Dictionary Comprehensions

A dictionary comprehension has the same structure as a list comprehension but uses braces and a `key: value` expression.

```
dict1: dict[int, int] = {x: x**2 for x in range(5)}
print(dict1)    # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

dict2: dict[int, int] = {key: value for key, value in dict1.items()
   if key > 2}
print(dict2)    # {3: 9, 4: 16}

dict3: dict[int, int] = {key: value for key, value in dict2.items()
   if value > 10}
print(dict3)    # {4: 16}
```

9.2.4 Dictionary Methods

Method	Description
`clear()`	Removes all the items from the dictionary.
`get(key, default)`	Returns the value for the key, or `default` (default `None`) if not found.
`items()`	Returns a view object of key-value pairs as tuples.
`keys()`	Returns a view object of all keys.
`pop(key, default)`	Removes the item and returns its value, or `default`; otherwise raises `KeyError`.
`popitem()`	Removes and returns the last inserted key-value pair as a tuple.
`update(other)`	Merges key-value pairs from `other`, updating existing keys and adding new ones.
`values()`	Returns a view object of all values.

```
students: dict[str, str] = { "21053124": "Tammy", "21543257": "Elvis"
   }
print(tuple(students.keys()))     # ('21053124', '21543257')
print(tuple(students.values()))   # ('Tammy', 'Elvis')
print(students.get("21053124"))   # Tammy
print(students.get("22222222"))   # None
print(students.pop("21053124"))   # Tammy
print(students)                   # {'21543257': 'Elvis'}
print(students.items())  # dict_items([('21543257', 'Elvis')])
students.clear()
print(students)                   # {}

dict1: dict[str, int] = {'a': 1, 'b': 2}
dict2: dict[str, int] = {'b': 3, 'c': 4}
merged: dict[str, int] = dict1 | dict2   # Merge with the | operator
print(merged)                            # {'a': 1, 'b': 3, 'c': 4}
```

9.2.5 Copying Dictionaries

As with lists, `dict2 = dict1` copies the reference, not the dictionary. A shallow copy can be made with a dictionary comprehension or the dictionary constructor `dict(dict1)`; a deep copy uses `copy.deepcopy(dict1)`. Shallow copies share references to nested objects, while deep copies create independent copies.

```
my_dict1: dict[tuple[int, int], int] = {(x, x + 1): x for x in
   range(10)}
my_tuple: tuple[int, int] = (5, 6)
print(my_dict1[my_tuple])   # 5
print(my_dict1[(1, 2)])     # 1
print(my_dict1[1, 2])  # Same as my_dict1[(1, 2)]

# my_dict2 = { [x, x+1]: x for x in range(10) }  # Not hashable
# my_set = { [1, 2, 3], [4] }                      # Error
```

Tuples as Keys

A tuple may be used as a dictionary key (or a set element) if all of its elements are immutable, whereas a list cannot, because lists are not hashable.

9.3 Choosing a Container Type

- **Lists**: use for an ordered sequence of items that can be modified, where the order of elements matters.
- **Tuples**: use for an ordered sequence that should not change after creation; tuples can serve as dictionary keys and iterate faster than lists.
- **Dictionaries**: use to store key-value pairs and perform fast lookups by key, ideal for mappings between items and for associating data with identifiers.

Summary

- A **tuple** is an ordered, immutable sequence; it cannot be changed after creation, but it may contain mutable objects.
- Tuples support the same sequence operations as lists (indexing, negative indexing, slicing, traversal, `len`/`min`/`max`/`sum`, comparison), except that slices cannot be assigned.
- Tuple methods are `count` and `index`; Python has no tuple comprehension – use `tuple()` with a generator.
- Automatic packing and unpacking create and split tuples, enabling multiple assignment and swapping.
- A **dictionary** stores key/value pairs with unique, hashable keys and supports fast lookups.
- Items are added or updated with `d[k] = v`, retrieved with `d[k]`, and removed with `del d[k]`; dictionaries have no positional indexing or slicing.
- Traversal uses `keys()`, `values()`, and `items()`; do not modify a dictionary while iterating it.
- Dictionary comprehensions build dictionaries concisely, and methods such as `get`, `pop`, `update`, and `clear` manage items.
- `dict2 = dict1` copies a reference; use a comprehension, the constructor, or `copy.deepcopy` to duplicate.
- Immutable tuples can serve as dictionary keys or set elements, unlike lists.

10 Recursion

10.1 Introduction

Recursion is a technique that leads to elegant solutions for problems that are difficult to solve using simple loops. It is the process of defining a solution to a problem in terms of a simpler version of itself, and many natural phenomena exhibit recursion.

A **recursive function** is a function that calls itself. It consists of two parts:

- The **base case(s)**: the conditions that stop the recursion.
- The **recursive case(s)**: the parts in which the function calls itself.

10.2 The Handshake Problem

Problem statement: there are n people in a room. If each person shakes hands once with every other person, what is the total number of handshakes?

For 1 person (A) there are 0 handshakes; for 2 people (A and B) there is 1; for 3 people (A, B, C) there are 3 (AB, AC, BC); and for 4 people (A, B, C, D) there are 6.

The trick is to see the pattern: adding one person to a group of $n-1$ people creates $n-1$ extra handshakes. Let $h(n)$ be the total number of handshakes among n people:

$h(1) = 0$ (base case); $h(2) = h(1) + 1 = 1$; $h(3) = h(2) + 2 = 3$; $h(4) = h(3) + 3 = 6$; and in general $h(n) = h(n-1) + (n-1)$.

$$h(n) = \begin{cases} h(n-1) + (n-1), & \text{if } n \geq 2 \\ 0, & \text{otherwise} \end{cases}$$

```
# Filename: handshake.py
def handshake(n: int) -> int:
    if n <= 1:                          # Base case
        return 0                        # No one to shake hands with
    return handshake(n-1) + (n-1)       # General case

def main() -> None:
    for i in range(5):
        print(f"{i} Person: {handshake(i)} handshake(s)")

if __name__ == "__main__":
    main()
```

The output lists 0 handshakes for 0 and 1 person, 1 handshake for 2 people, 3 handshakes for 3 people, and 6 handshakes for 4 people.

10.3 General Structure of a Recursive Function

A recursive function is implemented using an `if` or `if-else` statement that leads to different cases. One or more base cases (the simplest cases) stop the recursion, and every recursive call reduces the original problem, bringing it closer to a base case until that case is reached.

```
def <recursive_function_name>(<parameter list>):
    # Base case: condition to stop recursion
    if <base_case_condition>:
        return <base_case_value>
    # General case: call itself with modified parameters
    return <recursive_function_name>(modified_parameters)
```

10.4 The Factorial Problem

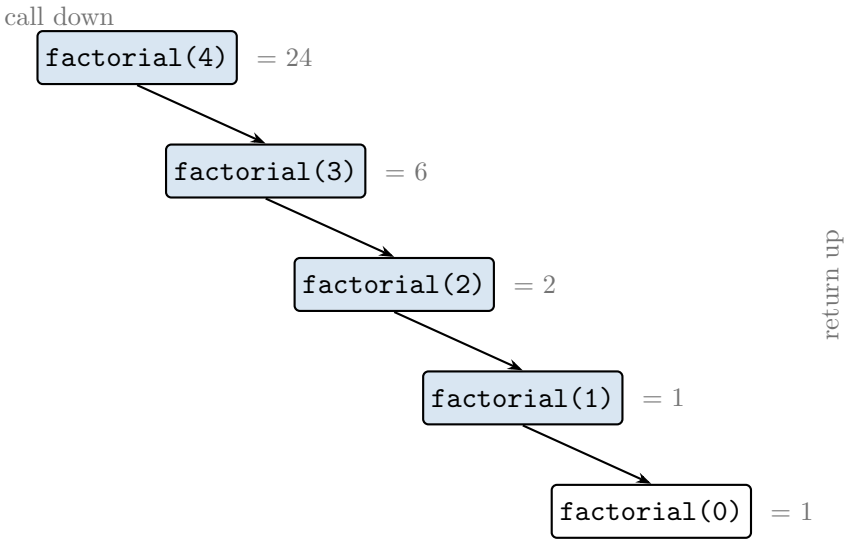
The factorial of n is defined as $n! = n \times (n-1)!$, where n is a non-negative integer.

$$f(n) = \begin{cases} n \times f(n-1), & \text{if } n > 0 \\ 1, & \text{if } n = 0 \end{cases}$$

```
# Filename: factorial_recursive.py
def factorial(n: int) -> int:
    if n == 0:                    # Base case
        return 1
    return n * factorial(n - 1)  # General case

def main() -> None:
    for i in range(6):
        print(f"{i}! = {factorial(i)}")

if __name__ == "__main__":
    main()
```



Tracing `factorial(3)`: since $3 \neq 0$, the function computes $3 \times factorial(2)$; then $2 \times factorial(1)$; then $1 \times factorial(0)$; then `factorial(0)` hits the base case and returns 1. The returns then unwind: $1 \times 1 = 1$, $2 \times 1 = 2$, and $3 \times 2 = 6$, so `factorial(3)` is 6.

Recursion Can Replace a Loop

It is simpler and more efficient to compute the factorial with a loop (using a `product` variable and a `while n > 0` loop). The recursive version is used here to demonstrate the concept of recursion; for certain problems a recursive solution leads to short and elegant code.

10.5 Fibonacci's Bunnies

Each new pair of bunnies becomes fertile at the age of one month, and each fertile pair produces a new pair every month; none of the bunnies die. Starting from one pair, the counts after successive months are 1, 1, 2, 3, 5, 8, ... – the **Fibonacci sequence**, in which each number is the sum of the two preceding numbers.

Let $F(n)$ be the number of pairs after n months:

$$F(n) = \begin{cases} 1, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ F(n-1) + F(n-2), & \text{if } n \geq 2 \end{cases}$$

```
# Filename: fib.py
def fib(n: int) -> int:
    if n == 0 or n == 1:           # Base cases
        return 1
    return fib(n - 1) + fib(n - 2) # Recursive case

def main() -> None:
    m: int = int(input("Enter month: "))
    num_bunnies: int = fib(m)
    print(f"The number of pairs of bunnies after {m} month(s) is
   equal to {num_bunnies} pair(s)")

if __name__ == "__main__":
    main()
```

For month 5 the answer is 8 pairs, for month 10 it is 89 pairs, and for month 20 it is 10946 pairs.

Repeated Work in Naive Fibonacci

Computing `fib(4)` by recursion expands into overlapping subproblems, so many intermediate values are recalculated. This repeated work makes the naive recursive Fibonacci inefficient for large n.

10.6 Recursion and Loops

	Loop	**Recursion**
Initialization	Initialize variables before the loop.	Pass initial values as parameters.
Condition	Check a condition at the beginning or end of each iteration.	Base case checks to stop further calls.
Iteration / Recursion	Update variables within the loop.	Call the function with updated parameters.
Termination	Exit when the condition is false.	Return a value when the base case is reached.

10.7 Stack Overflow and Its Handling

A **stack overflow** occurs when a recursive function exceeds the maximum call stack size, which can lead to a `RecursionError` in Python. Strategies to handle it include:

- **Base case**: ensure the base case is correctly defined and reachable (for example, return 1 when $n = 0$ in a factorial function).
- **Tail recursion**: make the recursive call the last operation, with no additional computation afterwards; some compilers can optimize this, although Python does not.
- **Iterative solutions**: convert the recursion into loops and an explicit stack.
- **Increase the recursion limit**: use `sys.setrecursionlimit(limit)`, being careful that a very large limit can lead to crashes.

10.8 Trade-offs and Caution

Recursion enables intuitive, straightforward solutions, but calling a function consumes more time and memory than adjusting a loop counter, so high-performance applications

rarely use it. In less demanding applications, recursion can be an attractive alternative to iteration for the right problems.

Infinite Recursion

Just as a loop can become infinite, recursion can create an infinite chain of calls. A recursive function with no base case (`return n * factorial(n - 1)`) or with a recursive call that never reaches the base case (`return n * factorial(n + 1)`) will recurse forever.

Designing Recursive Functions

Recursion decomposes a task into smaller subtasks, at least one of which is a simpler example of the same task. Always ensure that a recursive function contains at least one non-recursive branch (a base case), and that the recursive calls eventually lead to it – for example, $n! = n \times (n-1)!$ with the simplest task $1! = 1$.

Summary

- Recursion defines a solution in terms of a simpler version of itself; a recursive function calls itself.
- A recursive function has base case(s) that stop the recursion and recursive case(s) that call it with modified parameters.
- The handshake problem follows $h(n) = h(n-1) + (n-1)$ with $h(1) = 0$, solved by a simple recursive function.
- The factorial $n! = n \times (n-1)!$ with $0! = 1$ illustrates the call-and-return structure of recursion.
- Fibonacci's bunnies follow the Fibonacci sequence, computed by $F(n) = F(n-1) + F(n-2)$ with base cases $F(0) = F(1) = 1$.
- Recursion and loops are alternatives; recursion passes initial values as parameters and returns at the base case, while a loop updates variables until a condition fails.
- Stack overflow raises `RecursionError`; handle it with a correct base case, tail recursion, iteration, or a raised recursion limit.
- Recursion costs time and memory per call, so it suits problems where its elegance outweighs the overhead.
- A recursive function must always have a reachable base case to avoid infinite recursion.

11 NumPy

11.1 Why NumPy

NumPy (Numerical Python) is the core library for numerical and scientific computing in Python, providing a high-efficiency multidimensional array structure `np.ndarray`; libraries such as pandas and matplotlib build on it. An array holds one reference to one **contiguous block** of data, whereas a Python list holds references to full Python objects, so arrays use less memory and run faster.

What Is a NumPy Array

A **grid** of values, all of the **same data type**, indexed by a tuple of non-negative integers. The **rank** is the number of dimensions; the **shape** is a tuple giving the size along each dimension. Arrays are built from **nested** Python lists and read with square brackets.

A Small Performance Comparison

For one million elements, element-wise multiplication takes about 0.13 s with Python lists but only about 0.006 s with NumPy arrays. Arrays also allow `arr + 4`; a list raises `TypeError` for `ls + 4`.

11.2 Creating Arrays and Attributes

```
# Filename: array_basics.py
import numpy as np

a: np.ndarray = np.zeros((3, 3))       # also np.ones/full/eye
d: np.ndarray = np.arange(20, 61)      # 20, 21, ..., 60
e: np.ndarray = np.linspace(0, 1, 5)   # 5 evenly spaced values
f: np.ndarray = np.random.rand(3, 2)   # random floats in [0, 1)

arr: np.ndarray = np.array([[7, 8, 9], [10, 11, 12]])
print(arr.shape, arr.ndim, arr.size)   # (2, 3) 2 6
print(arr.dtype)                       # int64
x = np.array([3, 4], dtype=np.float32)  # set dtype explicitly
```

Creating and Describing Arrays

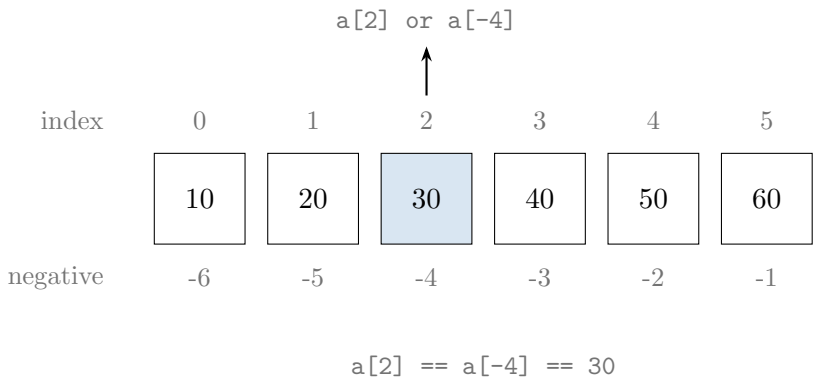
`np.array` builds an array from a (nested) list; `np.zeros`, `np.ones`, `np.full` and `np.eye` fill with constants or an identity; `np.arange` takes start/stop/step while `np.linspace` returns evenly spaced points; `np.random` makes random arrays. `ndim` is the rank, `shape` the per-dimension sizes, `size` the element count and `dtype` the element type.

11.3 Indexing and Negative Indexing

Indexing begins at 0; in a 2-D array you give one index per dimension, separated by a comma. A negative index counts from the end: `a[-1]` is the last element, `a[-2]` the second last.

```
# Filename: array_indexing.py
import numpy as np

# a has shape (3, 4):
# [[ 10  20  30  40] [ 50  60  70  80] [ 90 100 110 120]]
a: np.ndarray = np.array([[10, 20, 30, 40],
                          [50, 60, 70, 80],
                          [90, 100, 110, 120]])
print(a[0, 2])    # 30
print(a[-1, -1])  # 120   (last row, last column)
```

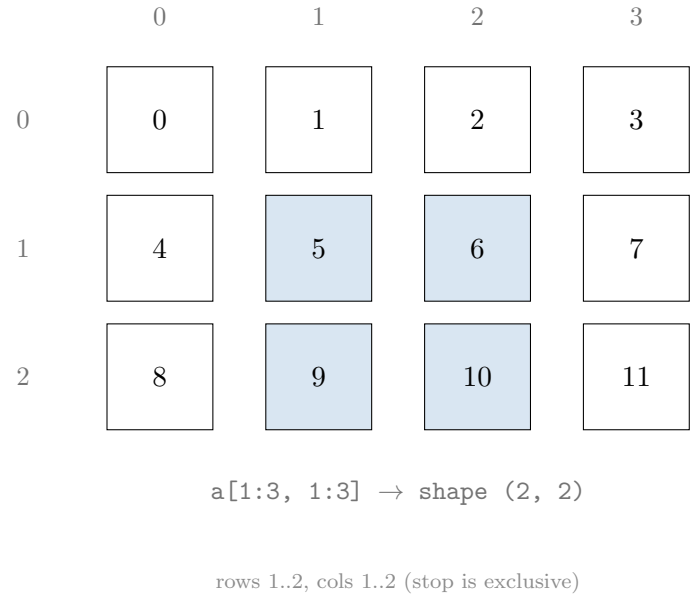


11.4 Slicing

A slice `a[i:j]` reads elements from index i up to, but **not including**, index j: the stop bound is **exclusive**. A slice is a **view**, so writing to it also changes the original.

```
# Rows 0-1 and columns 2-3; result has shape (2, 2)
b: np.ndarray = a[:2, 2:4]  # forms: a[:], a[i:j:n], a[::-1]
b[0, 0] = 99                # same data as a[0, 2]
print(a[0, 2])              # 99
```

For a 2-D array you slice each dimension. Mixing an integer index with a slice **lowers the rank**, while using only slices keeps the rank: `a[2, :]` has shape (4,) but `a[2:3, :]` keeps shape (1, 4).



11.5 Integer Array (Fancy) Indexing

Integer array indexing (also **fancy** or **advanced** indexing) selects arbitrary elements using arrays of indices.

```
# Filename:  integer_array_indexing_example .py
import numpy as np

a: np.ndarray = np.array([[7, 8], [9, 10], [11, 12]])
print(a[[0, 1, 2], [1, 0, 1]])   # [8 9 12]  (shape (3,))
```

```
print(a[[1, 1], [0, 0]])          # [9 9]     (element reused)
a2: np.ndarray = np.array([[13, 14, 15], [16, 17, 18],
                           [19, 20, 21], [22, 23, 24]])
b: np.ndarray = np.array([1, 0, 2, 1])   # one index per row
print(a2[np.arange(4), b])               # [14 16 21 23]
a2[np.arange(4), b] += 5                 # update those elements
```

11.6 Boolean Masking

Boolean array indexing extracts the elements meeting a condition: `a < 10` makes a Boolean array of the same shape, and using it as an index returns the matching elements.

```
# Filename: boolean_array_indexing_example.py
import numpy as np

a: np.ndarray = np.array([[7, 8], [9, 10], [11, 12]])
print(a[a < 10])                  # [7 8 9]
print(a[(0 < a) & (a < 10)])      # [7 8 9]  (AND; use | for OR)
```

Parenthesise Each Condition

Combine conditions with `&`, `|` and the tilde character, but wrap each one: write `a[(0 < a) & (a < 10)]`, not `a[0 < a & a < 10]`.

11.7 Reshaping and Expanding Dimensions

`reshape` changes the shape without changing the data (one dimension may be -1 to be inferred); `ravel` flattens to 1-D. `np.newaxis` (an alias for `None`) inserts a size-1 axis; `np.expand_dims(arr, axis)` does the same with an explicit axis.

```
# Filename: reshape_example.py
import numpy as np

r1: np.ndarray = np.arange(12).reshape(4, 3)   # shape (4, 3)
r2: np.ndarray = r1.ravel()                    # flattened (12,)

v: np.ndarray = np.array([10, 20, 30, 40, 50])
print(v[np.newaxis].shape, v[None].shape)   # (1, 5) (1, 5)
m: np.ndarray = np.array([[7, 8, 9], [10, 11, 12]])
print(np.expand_dims(m, axis=1).shape)    # (2, 1, 3)
```

newaxis vs expand_dims

Both add a size-1 axis and return a view; use `np.newaxis` inline, and `np.expand_dims` when the axis position comes from a variable.

11.8 View vs Copy

A **view** references the original data, so changes through it affect the original; a **copy** is a new object with the same data, so changes to it leave the original unchanged.

```
# Filename: view_copy_example.py
import numpy as np
import copy
```

```
arr: np.ndarray = np.array([1, 2, 3])
view_arr: np.ndarray = arr[1:3]               # a view
view_arr[0] = 10                              # also modifies arr

original: np.ndarray = np.array([1, 2, 3])
copy_arr: np.ndarray = copy.copy(original)    # a copy
copy_arr[0] = 10                              # original stays
```

Views include `arr[1:3]`, `reshape`, `transpose`, `view()` and `np.newaxis`; copies include `arr.copy()`, `np.copy(arr)`, `arr.astype(...)` and integer-array indexing.

11.9 Vectors, Matrices and Tensors

A **vector** is a 1-D array, a **matrix** is a 2-D array, and a **tensor** has three or more dimensions. Arithmetic operations act element by element, and are available both as operators and as NumPy functions.

```
# Filename: array_math_example.py
import numpy as np

x: np.ndarray = np.array([[2, 3], [4, 5]], dtype=np.float64)
y: np.ndarray = np.array([[6, 7], [8, 9]], dtype=np.float64)
print(x + y)       # or np.add(x, y)      -> [[8 10] [12 14]]
print(x * y)       # or np.multiply(x, y) -> [[12 21] [32 45]]
print(np.sqrt(x))  # element-wise square root

z: np.ndarray = np.array([[5, 10], [15, 20]])
print(np.mean(z), np.mean(z, axis=0), np.mean(z, axis=1))
# 12.5           [10. 15.]             [7.5 17.5]
# axis=0 reduces per column; axis=1 reduces per row
```

11.10 Dot Product and Matrix Multiplication

The **dot product** of two equal-size vectors is the sum of the products of corresponding elements; the result is a single **scalar**, and it is available as both a function and a method. **Matrix multiplication** uses `dot`, `matmul` or, since Python 3.5, the `@` operator.

```
# Filename: dot_product_example.py
import numpy as np

x: np.ndarray = np.array([[2, 4], [6, 8]])
y: np.ndarray = np.array([[1, 3], [5, 7]])
v: np.ndarray = np.array([8, 9])
w: np.ndarray = np.array([10, 11])
print(v.dot(w), np.dot(v, w)) # 179 179   (scalar)
print(x.dot(v))               # [52 120]           matrix/vector
print(v @ w)                  # 179       (@ operator)
print(x @ y)                  # [[22 34] [46 74]]  matrix/matrix
```

dot vs matmul

For rank-2 arrays `dot` and `matmul` agree, but for arrays of rank greater than 2 the results can differ. Prefer `matmul` or `@` for matrix multiplication.

11.11 Transpose

Transposing turns rows into columns and vice versa, using the `T` attribute, `np.transpose` or `transpose()`. For higher-dimensional arrays, `transpose` takes an `axes` permutation (such as `[2, 0, 1]`). A rank 1 array is unaffected: `v.T` of `np.array([1, 2, 3])` is still `[1 2 3]`.

```
# Filename: transpose_example.py
import numpy as np

a: np.ndarray = np.array([[5, 6], [7, 8]])
print(a.T)                # [[5 7] [6 8]]
print(np.transpose(a))    # [[5 7] [6 8]]
```

11.12 Broadcasting

Broadcasting lets NumPy combine arrays of different shapes in arithmetic by applying a smaller array repeatedly, instead of an explicit loop. Adding a vector `v` of shape (3,) to every row of a matrix `x` of shape (4, 3) works with a loop, but that is inefficient for a large matrix.

```
# Filename: broadcasting_example.py
import numpy as np

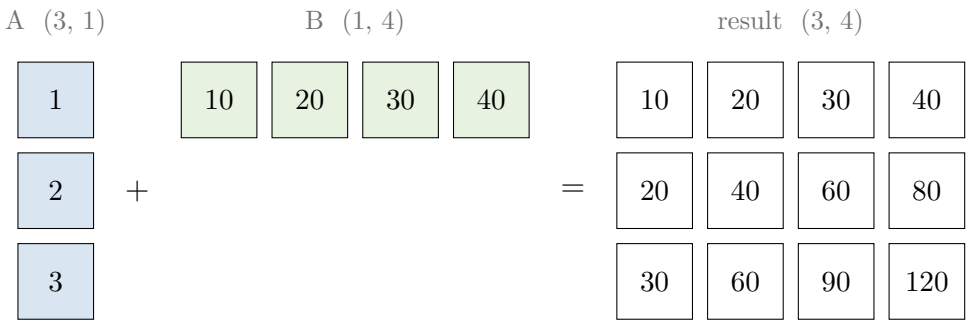
x: np.ndarray = np.array([[2, 3, 4], [5, 6, 7],
                          [8, 9, 10], [11, 12, 13]])
v: np.ndarray = np.array([2, 1, 2])
y: np.ndarray = x + v    # v broadcast over the rows of x
print(y)              # [[4 4 6] [7 7 9] [10 10 12] [13 13 15]]
```

Here `x` is (4, 3) and `v` is (3,); broadcasting lets the line behave as if `v` were (4, 3), each row a copy of `v`.

Broadcasting Rules

1. If the arrays differ in rank, prepend 1s to the shape of the lower-rank array until both shapes have equal length.
2. Two arrays are compatible in a dimension if their sizes are equal there, or if one of them has size 1.
3. They can be broadcast together if they are compatible in every dimension.

After broadcasting each array acts as if it had the element-wise maximum of the two shapes; a size-1 dimension behaves as if replicated.



The column and the row are stretched to a common shape.

Summary

- NumPy provides the fast `np.ndarray` grid of same-type values, described by rank and shape.
- Arrays are created with `np.array`, `np.zeros`, `np.ones`, `np.full`, `np.eye`, `np.arange`, `np.linspace` and `np.random`; `ndim`, `shape`, `size` and `dtype` describe them.
- 1-D and 2-D indexing, negative indexing and slicing (stop is exclusive) read subarrays; integer-array and Boolean indexing select arbitrary elements.
- `reshape`/`ravel` change shape; `np.newaxis` and `np.expand_dims` add size-1 axes; slices and reshapes are views while `copy` is independent.
- Element-wise arithmetic, `dot`/`matmul` and broadcasting make vector, matrix and tensor computation fast.

12 Object-Oriented Programming

12.1 Everything Is an Object

In Python every value is an **object**, including numbers, strings and other data types. Objects have an **identity**, a **type** and a **value**: the built-in `id()` returns the unique identifier of an object and `type()` returns its type, for example `type(3)` is `<class 'int'>`.

12.2 Objects and Classes

Object-Oriented Programming (OOP) involves using objects to create programs. An **object** represents an entity in the real world that can be distinctly identified, such as a student, a desk or a circle.

Identity, State and Behavior

- **Identity**: Python assigns each object a unique ID for identifying it at runtime.
- **State (attributes)**: represented by variables, for example the radius of a circle object.
- **Behavior (methods)**: actions performed by the object through functions, for example `getArea()` for a circle.

A **class** serves as a template, a blueprint and a data type for creating objects: it bundles their data fields (instance variables) together with their functionality (methods). An **instance** is an object of a class.

Instance Variables and Methods

- **Instance variables** belong to an object; they are public by default and accessed with the dot operator (.).
- **Instance methods** are defined with an extra first parameter `self`. No value is given for `self` when calling the method; Python provides it automatically.
- The `__init__` method is an **initializer** used to initialise the instance variables of objects.

12.3 Defining a Class and `__init__`

```
# Filename: circle.py
import math

class Circle:
```

```
    def __init__(self, radius: float):
        self.radius: float = radius    # an instance variable

    def area(self) -> float:
        return math.pi * self.radius ** 2

    def circumference(self) -> float:
        return 2 * math.pi * self.radius

def main() -> None:
    circleObj = Circle(10)      # create an object
    circleObj.radius = 100      # modify an instance variable
    print("Area:", circleObj.area())
    print("Circumference:", circleObj.circumference())

if __name__ == "__main__":
    main()
# Output: Area: 31415.926535897932
#         Circumference: 628.3185307179587
```

The general syntax is `class Name:` followed by `def __init__(self, args):` and other methods. An object is created with `obj = Class(args)`, and its members are read or modified with the dot operator, e.g. `obj.radius = 100`.

The self Parameter

`self` is a parameter that references the object itself. When a method is called on an object, `self` is set to that object, so the method can access the object's members (attributes and methods).

Instantiation

An object is created in memory using a **constructor**, and then the class's `__init__` method is called to initialise it. (In Python a constructor is created with the `__new__` method, which is outside this course.)

12.4 Instance Attributes vs Class Attributes

Two Kinds of Attributes

- An **instance attribute** is set through `self`, usually in `__init__`, and belongs to one object only; each object has its own copy.
- A **class attribute** is defined directly in the class body, outside any method, and is **shared** by all instances.

```
class Circle:
    count: int = 0    # class attribute, shared by all objects

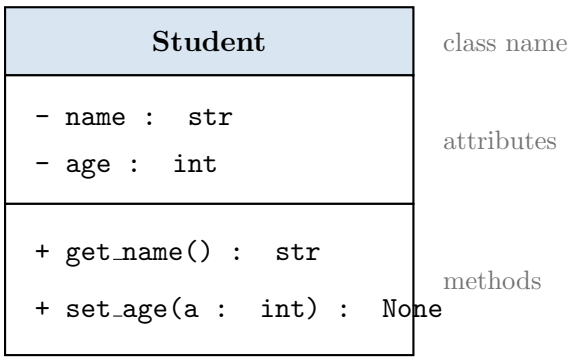
    def __init__(self, radius: float):
        self.radius = radius   # instance attribute per object
        Circle.count += 1
```

12.5 UML Class Diagrams

A **UML (Unified Modeling Language)** class diagram summarises a class by listing its data fields, constructors and methods.

UML Notation

- Data field: `dataFieldName: dataFieldType`
- Constructor: `ClassName(parameterName: parameterType)`
- Method: `methodName(parameterName: parameterType): returnType`
- The `__init__` method does not need to be listed.



12.6 Public, Protected and Private Members

By default all members in a Python class are **public**: any member can be accessed from outside the class. Access can be restricted by prefixing a member with underscores.

Access Conventions

- **Public**: no prefix; accessible everywhere.
- **Protected**: a single underscore, e.g. `_x`; accessible within the class and its sub-classes.
- **Private**: a double underscore, e.g. `__x`; accessible only from within the class.

By Convention Only

"By convention" means a responsible programmer should refrain from accessing or modifying members prefixed with _ or __ from outside their class. However, "if they do so, it will still work". Modern IDEs such as VS Code and PyCharm show a warning when you do.

```
# Filename: person.py
class Person:
    def __init__(self, name: str = "Tom", age: int = 18,
                 gender: str = 'M') -> None:
        self.__name: str = name    # private instance variables
        self.__age: int = age
        self.__gender: str = gender

    def print_info(self) -> None:
        print("Name: " + self.__name)
        print("Age: " + str(self.__age))
        print("Gender: " + self.__gender)

def main() -> None:
    desmond = Person("Haha", 18, "M")
    # print(desmond.__name)    # Error: private
    desmond.__name = "Desmond"  # Okay, but ...
    desmond.print_info()
```

Assignment Does Not Touch the Private Variable

From outside the class, `desmond.__name = "Desmond"` does not modify the private `__name` attribute; it silently creates a new, separate attribute. Use accessors and mutators to change the value safely.

12.7 Name Mangling

Name mangling is a mechanism the interpreter uses to modify the names of class attributes that begin with two underscores.

How Mangling Works

When an attribute name starts with two underscores (and does not end with two underscores), Python internally adds a single underscore and the class name as a prefix: an attribute `__myattribute` in a class `MyClass` becomes `_MyClass__myattribute`. Mangling happens at object creation, and the mangled name is used internally.

Mangling Is Not Privacy

Name mangling does not provide true privacy and is not a security feature; it is more of a convention to avoid naming conflicts. The value can still be read through its mangled name, e.g. `desmond._Person__name`.

12.8 Accessors and Mutators

Accessor and mutator methods are the proper way to read and modify protected or private instance variables that cannot be accessed from outside the class.

Getters and Setters

- An **accessor** (getter) returns the value of a private instance variable of an object.
- A **mutator** (setter) updates the value of a private instance variable of an object.

```
# Filename: person_w_accessors_mutators.py
class Person:
    def __init__(self, name: str = "Tom", age: int = 18,
                 gender: str = 'M') -> None:
        self.__name: str = name
        self.__age: int = age

    def getName(self) -> str:                # accessor (getter)
        return self.__name

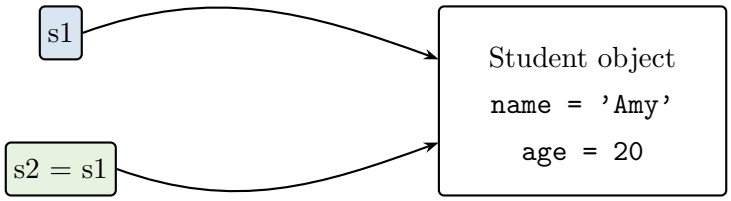
    def setName(self, name: str) -> None:    # mutator (setter)
        self.__name = name

def main() -> None:
    desmond = Person("Haha", 18, "M")
    print("Name: " + desmond.getName())
    desmond.setName("Desmond")
```

12.9 Assignment Copies the Reference, Not the Object

```
# Filename: circle_shallow_copy.py
circleObj1: Circle = Circle(10)
circleObj2: Circle = Circle(20)
circleObj1 = circleObj2   # copies reference, not contents
```

After the assignment, `circleObj1` and `circleObj2` refer to the **same** object. The circle with radius 10 is no longer referenced and can be garbage-collected. This is a **shallow copy**: the two names share one object. To copy the value instead, copy the attribute from one object to the other with `circleObj1.radius = circleObj2.radius`.



Assignment copies the reference, not the object.

Two Names, One Object

`circleObj1 = circleObj2` does **not** create a second circle. Both names point to one object, so a change through one name is visible through the other. Assign attribute values if you want an independent copy.

Summary

- In Python everything is an object with an identity (via `id()`), a type (via `type()`) and a value.
- A class is a template/data type for objects; an object is an instance of a class, with identity, state (attributes) and behavior (methods).
- `__init__` is the initializer and `self` references the object itself; Python supplies `self` automatically.
- Instance attributes belong to one object, while class attributes are shared by all instances.
- UML class diagrams describe data fields, constructors and methods.
- Members are public by default; a single underscore marks a protected member and a double underscore marks a private member, and name mangling rewrites the latter as `_ClassName__attr`.
- Accessors (getters) read and mutators (setters) modify private instance variables, and assignment copies a reference, not an object.

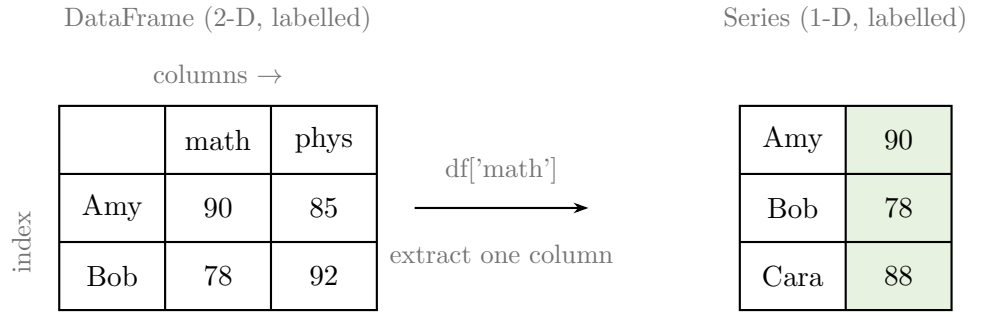
13 Pandas

13.1 Introduction

Pandas is a powerful Python library for **data manipulation** and **data analysis**, particularly suited to **structured data**. It provides two primary data structures, the **Series** (1-D) and the **DataFrame** (2-D). Key functionalities include data filtering, aggregation, reshaping with pivot tables, handling missing values, and exploratory data analysis. Pandas is built on top of NumPy, adding data alignment and intuitive indexing. This course uses version 2.3.1.

Series and DataFrame

Both structures attach data to **labels** (an **index**). A Series is a 1-D labelled array; a DataFrame is a 2-D labelled table whose columns share one common row index.



A DataFrame is a dictionary-like collection of Series sharing one index.

13.2 The Series

A **Series** is a one-dimensional labelled array that can hold any data type (integers, strings, floating point numbers, etc.). Each element has an **index**, which can be numeric or string labels. A Series can be created from a list, a dictionary, or a NumPy array, and supports element access, mathematical operations, and applying functions to the whole array.

13.2.1 Creating a Series

```
# Filename: series_create.py
import pandas as pd

s1 = pd.Series([10, 20, 30, 40])            # Default index
s2 = pd.Series([10, 20, 30, 40],            # Custom labels
               index=['first', 'second', 'third', 'fourth'])
data = {'first': 10, 'second': 20, 'third': 30, 'fourth': 40}
s3 = pd.Series(data)                        # From a dictionary
print(s1); print(s2); print(s3)
```

13.2.2 Accessing and Slicing

Label-based access uses `Series.loc[]` (the explicit index); position-based access uses `Series.iloc[]` (the integer position, like NumPy). The `.values` attribute returns a NumPy array and `.index` returns the labels. In a slice, `loc` **includes** both endpoints, while `iloc` **excludes** the stop.

```
# Filename: series_access_slicing.py
import pandas as pd

s = pd.Series([10, 20, 30, 40, 50, 60],
              index=['first', 'second', 'third',
                     'fourth', 'fifth', 'sixth'])

print(s.loc['second'])   # By label -> 20
print(s.iloc[2])         # By position -> 30
print(s.values)          # NumPy array
```

```
print(s.index)           # Index object
print(s.loc['third':'fifth'])  # Both endpoints included
print(s.iloc[2:5])             # Stop position excluded
```

loc slicing and missing labels

`s.loc['third':'fifth']` includes `'fifth'`, but `s.iloc[2:5]` stops before position 5. If you pass a label that is not in the index, `loc` raises a `KeyError`; `iloc` only uses positions. `.loc[]` and `.iloc[]` can also assign new values.

13.2.3 Masking

Masking creates a boolean Series where `True` marks the elements that satisfy a condition; the mask can then select or modify those elements.

```
# Filename: series_masking.py
import pandas as pd

s = pd.Series([10, 15, 25, 5, 30],
              index=['a', 'b', 'c', 'd', 'e'])

mask = (s > 10) & (s < 30)  # Values between 10 and 30
print(mask)                 # Boolean Series
s[mask] = 0                 # Modify elements where True
print(s)
```

Combining conditions

Wrap each comparison in parentheses and combine them with `&`, `|`, and `~`. The Python keywords `and`/`or` do not work on a Series, because they try to reduce the whole array to one boolean.

13.3 The DataFrame

A **DataFrame** is a powerful 2-dimensional data structure similar to a spreadsheet or SQL table. It consists of rows and columns, where each column is a Series that shares the same index, and each column has a unique name.

13.3.1 Creating a DataFrame

Pass a dictionary whose keys are column names and whose values are Series that share the index. Series with different indexes align by label; unmatched labels become `NaN`.

```
# Filename: dataframe_create.py
import pandas as pd

idx = ['Product A', 'Product B', 'Product C']
sales = pd.Series([100, 150, 200], index=idx)
cost = pd.Series([80, 90, 120], index=idx)
df = pd.DataFrame({'Sales': sales, 'Cost': cost})
print(df)
```

Other common sources are a list of dictionaries (one dictionary per row), a dictionary of key-list pairs, and a 2-D NumPy array with explicit column names.

```
# Filename: dataframe_create_sources.py
import numpy as np
```

```
import pandas as pd

# A list of dictionaries; each dictionary is one row
rows = [{'name': 'Emma', 'age': 28, 'city': 'Beijing'},
        {'name': 'Liam', 'age': 32, 'city': 'Shanghai'}]
df1 = pd.DataFrame(rows, index=['row1', 'row2'])

# A dictionary of key-list pairs; each key is one column
data = {"Name": ["Emma", "Liam"], "Age": [28, 32]}
df2 = pd.DataFrame(data)

# A 2-D NumPy array with named columns and rows
arr = np.array([[28, 32], [1, 2]])
df3 = pd.DataFrame(arr, columns=['Beijing', 'Shanghai'],
                   index=['Age', 'ID'])
print(df1); print(df2); print(df3)
```

13.3.2 Inspecting a DataFrame

```
# Filename: dataframe_inspect.py
import pandas as pd

df = pd.read_csv('./data/employees.csv')

print(df.head())     # First five rows
print(df.tail())     # Last five rows
print(df.shape)      # (rows, columns)
print(df.info())     # dtypes and non-null count per column
print(df.describe()) # Summary statistics for numeric columns
```

13.3.3 Selecting Columns, Rows and Cells

Access a column with `df["name"]` (returns a Series), and `.columns` or `.index` for the labels. Access a single row with `loc` (label) or `iloc` (position); it returns a Series indexed by the column names. Do not mix label-based and position-based indexing in one selection.

```
# Filename: dataframe_access.py
import pandas as pd

idx = ['A', 'B', 'C']
df = pd.DataFrame({'City': pd.Series(['Beijing', 'Shanghai',
                                      'Guangzhou'], index=idx),
                   'Population': pd.Series([21542000, 24183300,
                                            14904000], index=idx),
                   'Area (sq km)': pd.Series([16410.54, 6340.5,
                                              7434.4], index=idx)})
print(df["City"])                          # A column (Series)
print(df.loc['A'])                         # A row by label
print(df.iloc[0])                          # A row by position
print(df.loc['B':'C', 'Population':])      # Rows B..C
```

Masking selects rows by a condition and can be combined with slicing to keep chosen columns. **Fancy indexing** (advanced indexing) selects rows and/or columns by passing lists of labels, and can also assign to the selection.

```
# Filename: dataframe_mask_fancy.py
import pandas as pd

idx = ['A', 'B', 'C']
df = pd.DataFrame({'City': pd.Series(['Beijing', 'Shanghai',
                                      'Guangzhou'], index=idx),
                   'Population': pd.Series([21542000, 24183300,
                                            14904000], index=idx),
                   'Area (sq km)': pd.Series([16410.54, 6340.5,
                                              7434.4], index=idx)})
# Masking: filter rows, then slice the columns to keep
mask = (df['Population'] > 10000000) & (df['Area (sq km)'] < 7000)
print(df.loc[mask, 'Population':])

# Fancy indexing: pick rows and columns by lists of labels
print(df.loc[['A', 'C'], ['City', 'Population']])
df.loc[['A', 'C'], ['City', 'Population']] = ['Updated City', 0]
print(df.loc[['A', 'C'], ['City', 'Population']])
```

Changes that silently disappear

`df.drop(columns=[...])` and `df.rename(...)` **return a copy**; the original DataFrame is unchanged unless you reassign, e.g. `df = df.drop(columns=['x'])` or pass `inplace=True`. Adding a column with `df['new'] = ...` does modify the DataFrame in place.

13.4 Handling Missing Values

Missing values use the sentinel values `None` (a Python object) or `np.nan` (a float, faster for numerical work). Pandas supports both and converts between them when needed. `isnull()` (alias `isna()`) returns a boolean mask that is `True` where a value is missing; `notnull()` returns the opposite.

```
# Filename: handle_missing.py
import numpy as np
import pandas as pd

s = pd.Series([21540000, None, 8000000, np.nan],
              index=['Shanghai', 'Beijing',
                     'Chongqing', 'Guangzhou'])
print(s.isnull())          # True where missing
print(s.notnull())         # True where present
print(s.dropna())          # Remove missing entries
print(s.fillna(s.mean()))  # Replace NaN by the mean
```

dropna() removes whole rows

`df.dropna()` drops every row that contains at least one missing value, which can discard far more data than intended. Check `df.isnull().sum()` first, and prefer `fillna()` when dropping whole rows is too aggressive.

13.5 Grouping and Aggregation

Aggregate functions (`mean()`, `std()`, `min()`, `max()`, `sum()`) return a single value on a Series and one value per column on a DataFrame. `groupby()` splits rows by a key and

returns a DataFrameGroupBy object, which you can iterate or aggregate; the grouping key becomes the result index until you call `reset_index()`.

```
# Filename: dataframe_groupby.py
import pandas as pd

df = pd.DataFrame({'Department': ['HR', 'IT', 'HR', 'IT'],
                   'Salary': [50000, 60000, 55000, 70000],
                   'Experience': [2, 5, 3, 7]})
print(df[['Salary', 'Experience']].mean())  # Column-wise
grouped = df.groupby('Department')
for key, group in grouped:      # Iterate over the groups
    print(key); print(group)
print(grouped.mean().reset_index())  # Key back as a column
```

13.6 Reading and Writing CSV Files

Load a DataFrame with `pd.read_csv()`: `sep` sets the delimiter, `skiprows` skips lines, and `na_values` lists strings treated as missing. The header is read from the first line, dtypes are inferred, and empty fields become `NaN`. Save with `to_csv()`; `index=False` avoids writing the row index as a column.

```
# Filename: csv_io.py
import pandas as pd

df = pd.read_csv('./data/employees.csv',
                 sep=',', skiprows=0,
                 na_values=['N/A', 'Missing'])
print(df)

df.to_csv('./data/employees2.csv', sep=',', index=False)
```

Warm-up: filter and summarize

Given `df` with columns `Department` and `Salary`, print the average salary of the `IT` department.
Solution. Mask the rows and take the column mean: `df.loc[df['Department'] == 'IT', 'Salary'].mean()`. Grouping gives all departments at once: `df.groupby('Department')['Salary'].mean()`.

Quick check

1. What is the difference between `loc` and `iloc` slicing?
2. Why does adding two Series with different labels produce `NaN`?
3. What does `df.dropna()` do to a DataFrame?
4. How do you restore the grouping key as a column after `groupby`?

Summary

- Pandas provides the 1-D **Series** and the 2-D **DataFrame**, both built on NumPy and indexed by labels.
- Create DataFrames from Series, lists of dictionaries, dictionaries of key-list pairs, NumPy arrays, or CSV files; inspect them with `head`, `tail`, `shape`, `info` and `describe`.
- Select with `loc` (labels, stop included) and `iloc` (positions, stop excluded); a single column or row returns a Series.
- Filter with boolean **masking**, **fancy indexing**, and slicing.

- Handle missing values with `isnull`/`isna`/`notnull`, `dropna` and `fillna`.
- `groupby()` splits and aggregates; `reset_index()` restores the key column.
- Read and write CSV files with `pd.read_csv()` and `to_csv()`.

14 Matplotlib

14.1 Introduction

Matplotlib is a comprehensive library for creating **static**, animated, and interactive visualizations in Python. This course uses version 3.10.3 and focuses on **static** visualizations.

14.2 The Pyplot and Object-Oriented Interfaces

Pyplot (`matplotlib.pyplot`, imported as `plt`) is a stateful, MATLAB-like API that acts on the current figure. The workflow is: import the module, prepare the data, call `plt.plot()`, enhance the plot with `plt.title()`, `plt.xlabel()` and `plt.ylabel()`, then display it with `plt.show()`.

```
# Filename: pyplot_steps.py
import matplotlib.pyplot as plt

x = [0, 1, 2, 3]
y = [0, 2, 4, 6]

plt.plot(x, y)                  # Create the plot
plt.title("A Simple Plot")      # Enhance the visualization
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()                      # Display the plot
```

The **object-oriented (OO) interface** creates explicit `Figure` and `Axes` objects and calls methods on them. This course uses the OO interface because it is clearer when a figure holds several plots.

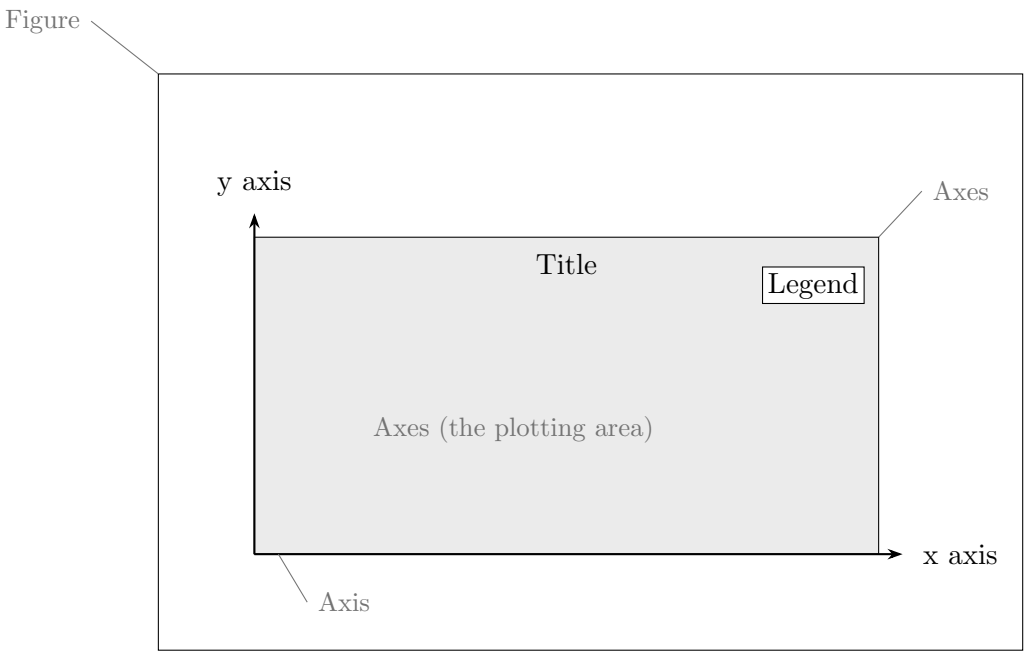
Pyplot versus object-oriented

`plt.plot(...)` acts on the currently active axes and is fine for a quick single plot. `fig, ax = plt.subplots(); ax.plot(...)` is explicit and scales to many subplots. Mixing the two styles on one figure is a common source of confusion; pick one style and stay with it.

14.3 Figures, Axes and Axis

A Matplotlib figure is composed of several key elements:

- **Figure**: the primary container of all visual elements, the canvas of the whole plot.
- **Axes**: the region within the figure where data is drawn; one figure can contain multiple axes.
- **Axis**: defines the horizontal (x-axis) and vertical (y-axis) dimensions, including their limits, tick marks, and labels.
- **Lines and markers**: lines connect data points to show trends, and markers highlight individual points (important in scatter plots).
- **Title and labels**: the title gives overall context, and axis labels clarify what each axis represents.



Use `fig, ax = plt.subplots(figsize=(5, 3))` to create a new Figure with its Axes (the width and height are in inches); give the number of rows and columns for a grid of subplots.

The current-figure trap

With Pyplot, a call such as `plt.hist(...)` draws on the current axes, which is easy to lose track of. The OO style removes the ambiguity: always call the drawing method on the intended `ax`.

14.4 Pairwise Data Visualization

Pairwise techniques cover (x, y) coordinates, tabular data, and functions of the form f(x) = y. The main methods are `plot(x, y)`, `scatter(x, y)`, `bar(x, height)`, `stem(x, y)`, `fill_between(x, y1, y2)`, `stackplot`, and `stairs`.

14.4.1 Line Plots with plot

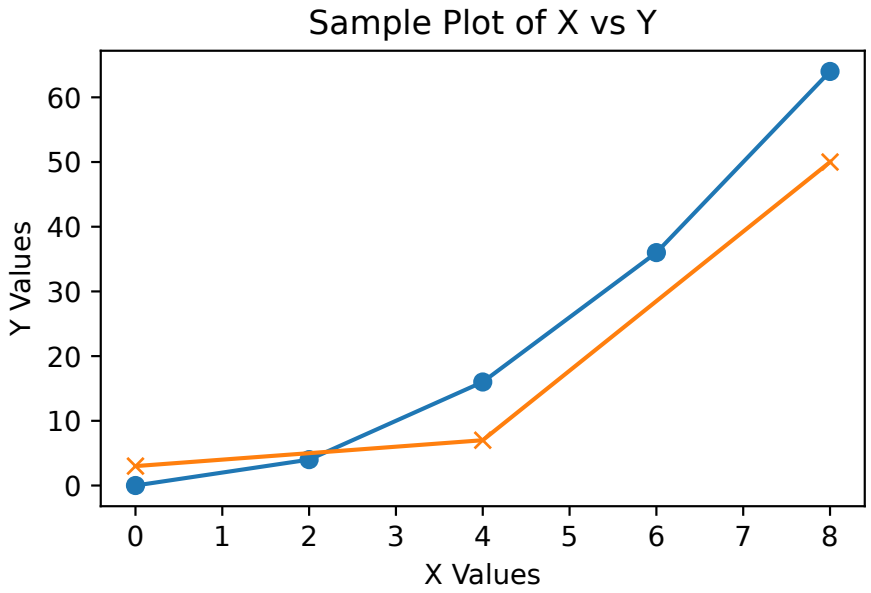
`ax.plot(x, y)` connects the points given by the x- and y-coordinate sequences. Customize markers with `marker`, add a legend entry with `label`, and set the axis names and title with `set_xlabel()`, `set_ylabel()` and `set_title()`. A single call can also draw a sequence of points or segments that share properties such as color and line style.

```
# Filename: line_curve_draw.py
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(5, 3))
# Line through (0,0), (2,4), (4,16), (6,36), (8,64)
ax.plot([0, 2, 4, 6, 8], [0, 4, 16, 36, 64],
        marker='o', label="Data Points 1")
# A sequence of markers only (no connecting line)
ax.plot([0, 4, 8], [3, 7, 50], linestyle="",
        marker='x', label="Data Points 2")

ax.set_xlabel("X Values")
ax.set_ylabel("Y Values")
```

```
ax.set_title("Sample Plot of X vs Y")
ax.legend(loc=(1.1, 0.5))   # Place the legend outside
plt.show()
```



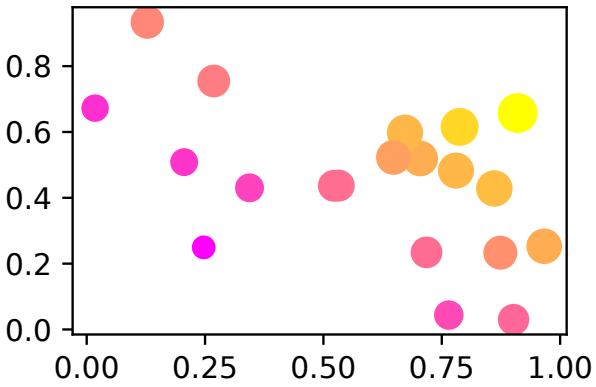
14.4.2 Scatter Plots

Use `ax.scatter(x, y)` to draw a set of individual points and give them custom properties. The `cmap` parameter selects a colormap, and `s` sets the marker size as an area in square points.

```
# Filename: scatterplot.py
import numpy as np
import matplotlib.pyplot as plt

x = np.random.rand(20)
y = np.random.rand(20)
colors = x + y
area = 100 * (x + y)

fig, ax = plt.subplots(figsize=(3, 2))
# cmap selects a colormap, s sets the marker area
ax.scatter(x, y, c=colors, cmap="spring", s=area)
plt.show()
```



plot versus scatter

`plot` draws a connected line by default; set the line style to empty (`linestyle=""`) to keep only markers. `scatter` draws independent points, so it can vary the color and size of each point separately.

14.4.3 Subplots

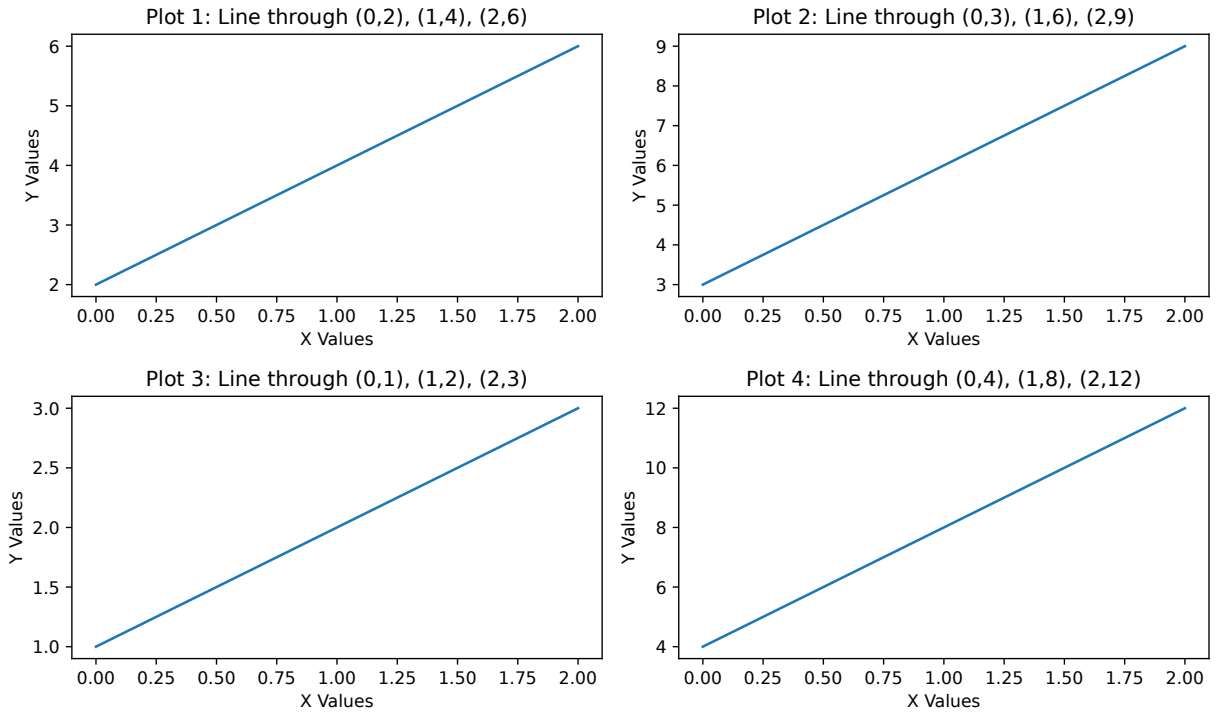
To draw several plots in one figure, call `plt.subplots(rows, cols)` with the numbers of rows and columns, then index the returned axes array with NumPy indexing.

```
# Filename: line_plot_draw_multiples.py
import matplotlib.pyplot as plt

fig, ax = plt.subplots(2, 2, figsize=(10, 6))

ax[0, 0].plot([0, 1, 2], [2, 4, 6]); ax[0, 0].set_title("Plot 1")
ax[0, 1].plot([0, 1, 2], [3, 6, 9]); ax[0, 1].set_title("Plot 2")
ax[1, 0].plot([0, 1, 2], [1, 2, 3]); ax[1, 0].set_title("Plot 3")
ax[1, 1].plot([0, 1, 2], [4, 8, 12]); ax[1, 1].set_title("Plot 4")

plt.tight_layout()   # Avoid overlapping labels
plt.show()
```



14.4.4 A Sequence of Points or Segments

A single `plot` call can also draw a sequence of points or segments that share the same properties (such as size, color and width); pass `linestyle=""` to keep only the markers.

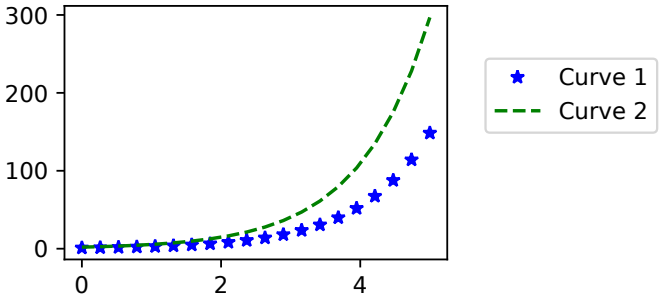
```
# Filename: sequence_points_segments_draw.py
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 5, 20)
y = np.exp(x)

fig, ax = plt.subplots(figsize=(3, 2))

ax.plot(x, y, c="blue", linestyle="",
        marker='*', label="Curve 1")
ax.plot(x, 2*y, c="green",
        linestyle="--", label="Curve 2")

# Specify the legend position with relative position
ax.legend(loc=(1.1, 0.5))
plt.show()
```



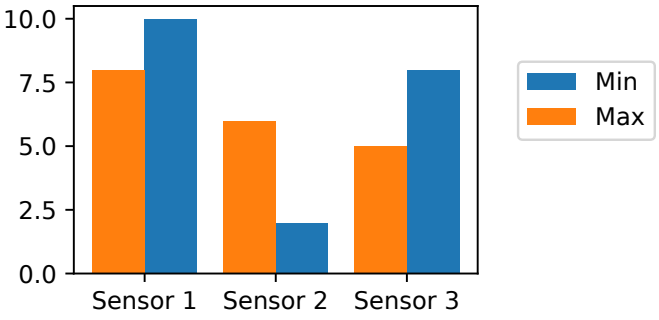
14.4.5 Bar Charts

`ax.bar(pos, height)` draws vertical bars (or `ax.barh` for horizontal). Positions and heights are matching lists, and `tick_label` assigns text labels to the x-axis ticks. To group bars side by side, shift the two groups horizontally by half the bar width.

```
# Filename: barchart.py
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(3)
width = 0.4
heightMin = [10, 2, 8]
heightMax = [8, 6, 5]
labels = ["Sensor 1", "Sensor 2", "Sensor 3"]

fig, ax = plt.subplots(figsize=(3, 2))
ax.bar(x + width / 2, heightMin, width=width, label="Min")
ax.bar(x - width / 2, heightMax, width=width, label="Max")
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend(loc=(1.1, 0.5))
plt.show()
```



14.5 Statistical Distributions

These methods depict the distribution of one or more variables: `errorbar(x, y, yerr, xerr)`, `hist(x)`, `boxplot(X)`, `violinplot(D)`, `eventplot(D)`, `hist2d(x, y)`, `hexbin(x, y, C)`, `pie(x)`, and `ecdf(x)`.

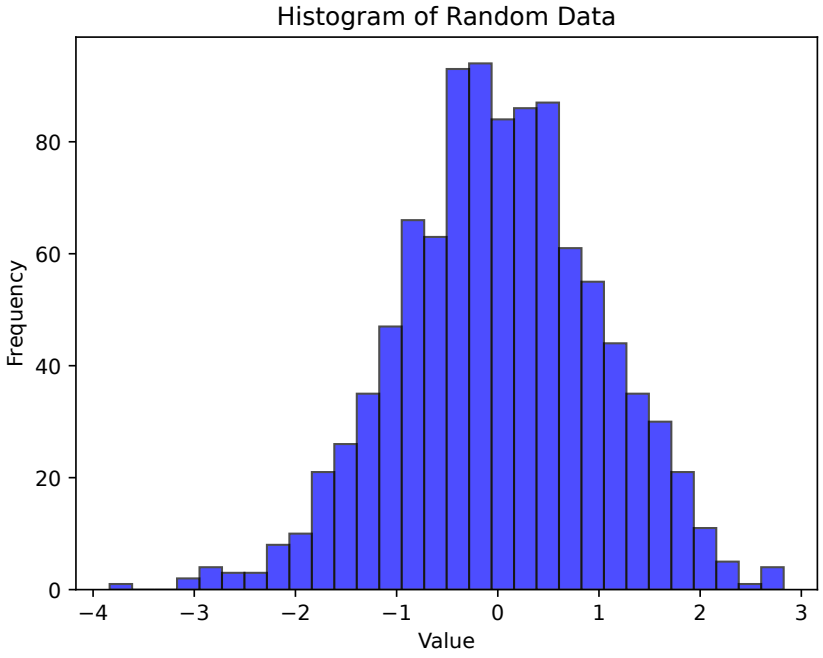
14.5.1 Histograms

`hist(x)` shows the frequency distribution of a one-dimensional dataset. The `bins` parameter controls the granularity, and `color`, `alpha` and `edgecolor` refine the appearance.

```
# Filename: histogram.py
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(1000)

plt.hist(data, bins=30, color='blue',
         alpha=0.7, edgecolor='black')
plt.title("Histogram of Random Data")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



Bins matter

The number of `bins` strongly influences how a histogram looks: too few hide structure and too many make it noisy. The data is unchanged; only the visual grouping differs.

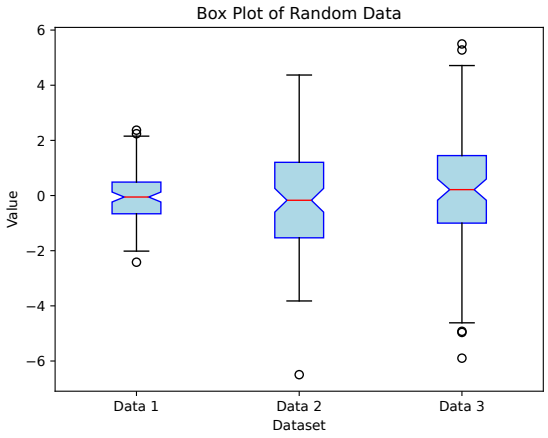
14.5.2 Box Plots

`boxplot(X)` summarizes a distribution through its quartiles, showing the central tendency, variability, and potential outliers. Pass a one-dimensional array for one box, or a list of arrays for several boxes. Parameters such as `notch`, `vert` and `patch_artist` control the appearance.

```
# Filename: boxplot.py
import matplotlib.pyplot as plt
import numpy as np

data = [np.random.normal(0, std, 100) for std in range(1, 4)]

plt.boxplot(data, notch=True, patch_artist=True,
            boxprops=dict(facecolor='lightblue',
                          color='blue'),
            medianprops=dict(color='red'))
plt.title("Box Plot of Random Data")
plt.xticks([1, 2, 3], ['Data 1', 'Data 2', 'Data 3'])
plt.show()
```



Reading a box plot

The box spans the first to the third quartile (the interquartile range), the line inside is the median, the whiskers extend to the most extreme non-outlier points, and points beyond the whiskers are outliers. The box is not a probability density.

14.6 Contour Plots

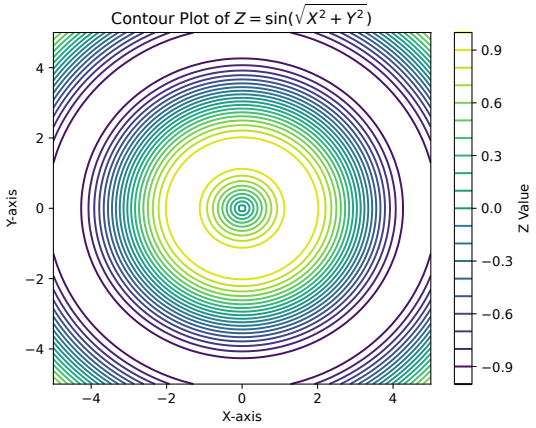
`contour(X, Y, Z)` represents three-dimensional data in two dimensions with contour lines. Build a grid with `np.meshgrid`: 2-D arrays for the x and y coordinates and a matching 2-D array of z values. `levels` sets the number of contour lines (or specific z values) and `cmap` chooses a colormap.

```
# Filename: contour.py
import numpy as np
```

```
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

plt.contour(X, Y, Z, levels=20, cmap='viridis')
plt.colorbar(label='Z Value')
plt.title("Contour Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

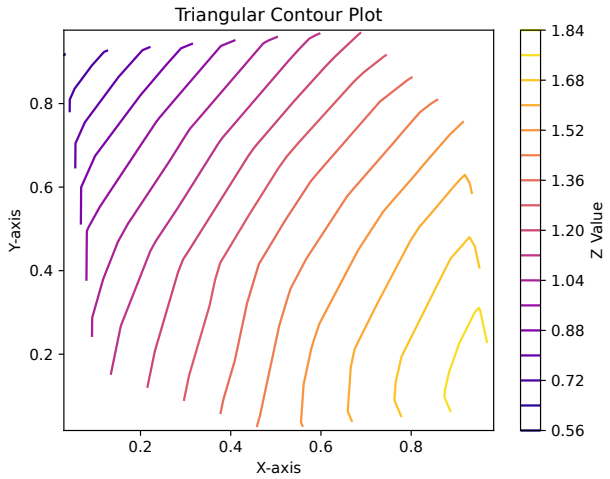


For irregularly spaced data, `tricontour()` works with a `Triangulation` built from the point coordinates instead of a rectangular mesh.

```
# Filename: tricontour.py
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.tri import Triangulation

x = np.random.rand(30); y = np.random.rand(30)
z = np.sin(x) + np.cos(y)
triang = Triangulation(x, y) # Create triangulation

plt.tricontour(triang, z, levels=14, cmap='plasma')
plt.colorbar(label='Z Value')
plt.title("Triangular Contour Plot")
plt.xlabel("X-axis"); plt.ylabel("Y-axis")
plt.show()
```



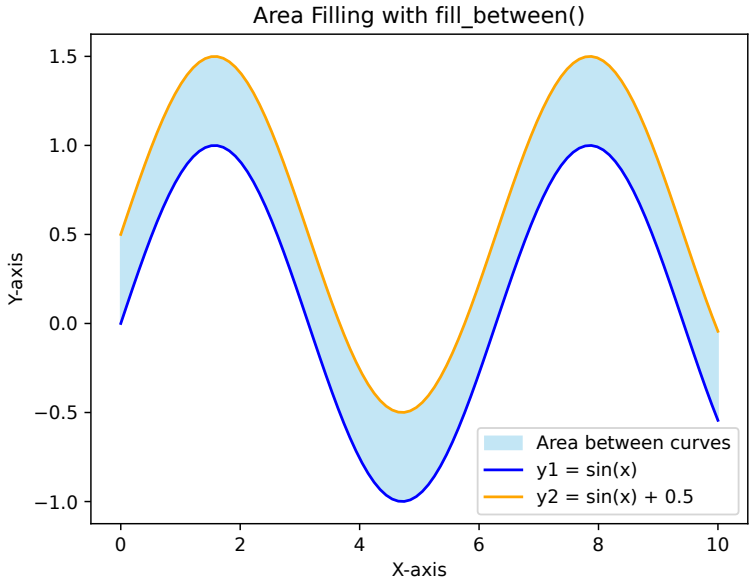
14.7 Filling the Area Between Curves

The method `fill_between(x, y1, y2)` fills the region between two horizontal curves. The x-coordinates and the two sets of y-coordinates define the boundaries; `color`, `alpha` and `label` customize the filled area.

```
# Filename: area_filling.py
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.sin(x) + 0.5

plt.fill_between(x, y1, y2, color='skyblue',
                 alpha=0.5, label='Area between curves')
plt.plot(x, y1, label='y1 = sin(x)', color='blue')
plt.plot(x, y2, label='y2 = sin(x) + 0.5', color='orange')
plt.title("Area Filling with fill_between()")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()
```



14.8 Saving a Figure to a File

Save a figure in many formats (JPEG, PNG, PDF, EPS, etc.) with `fig.savefig()`; call it before `plt.show()`, because showing may close or reset the figure.

```
# Filename: save_plot.py
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(3, 2))
ax.plot([0, 1, 2], [2, 4, 6])
ax.plot([0, 1, 2], [3, 6, 9])

# Save before showing; PDF keeps the plot as vector graphics
fig.savefig("test.pdf")
plt.show()
```

Prefer a vector PDF

Saving as PDF (or EPS/SVG) keeps lines and text as **vector** graphics, so the plot stays sharp at any zoom and print size. Raster formats such as PNG are resolution-limited; when a raster image is required, raise `dpi` (for example `fig.savefig("test.png", dpi=300)`).

14.9 Reading Data From Files

Plot data usually comes from a file:

- numeric data: `np.loadtxt` (or `np.genfromtxt`)
- tabular data: `pd.read_csv`

Pass the resulting arrays to the plotting methods.

```
# Filename: plot_from_file.py
import numpy as np
import matplotlib.pyplot as plt

# Read a two-column numeric file (one x,y pair per line)
data = np.loadtxt("data.txt", delimiter=",")
x, y = data[:, 0], data[:, 1]
```

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.show()
```

Mismatched lengths

`ax.plot(x, y)` and `ax.scatter(x, y)` require x and y to have the same length. Loading the wrong column raises `ValueError: x and y must have same first dimension`. Print `x.shape` and `y.shape` to check.

Worked example: visualize a distribution

Given a file column of 500 measurements, produce a histogram and a box plot and save the result as a vector file.
Solution. Read the column with `np.loadtxt`, call `plt.hist(values, bins=30)` and `plt.boxplot(values)`, then use `fig.savefig("report.pdf")` before `plt.show()`.

Quick check

1. What is the difference between the Figure, the Axes, and the Axis?
2. How do you create a figure with several plots in a grid?
3. Which parameter customizes the marker appearance in `plot`?
4. How do you fill the area between two curves?
5. Why is saving as a vector PDF often preferred over PNG?

Summary

- Matplotlib creates static, animated, and interactive visualizations; this course focuses on static plots (version 3.10.3).
- Use the object-oriented interface (`fig, ax = plt.subplots()`) for clarity and multi-panel figures.
- A figure contains Axes; each Axes has an x-axis and y-axis with limits, ticks, and labels.
- Pairwise plots: `plot` (lines), `scatter` (points), `bar` (categories), `fill_between` (area bands).
- Distribution plots: `hist` (bins) and `boxplot` (quartiles and outliers).
- Gridded data: `contour` (and `tricontour` for irregular grids).
- Add titles and labels with `set_title`, `set_xlabel`, `set_ylabel`, and place a legend with `legend`.
- Create multiple plots with `subplots(rows, cols)` and avoid overlaps with `tight_layout()`.
- Save with `fig.savefig()` (prefer a vector PDF) before `plt.show()`; read plot data from files with `np.loadtxt`.